基于同星 TSMaster COM API 的汽车 ECU 自动化测试系统开发指南

1. 系统架构与开发环境搭建

1.1 TSMaster 平台核心功能概述

TSMaster 是由上海同星智能科技有限公司开发的汽车电子开发平台软件，采用软硬件解耦设计理念，支持快速迭代和持续功能扩展。该平台保持每周更新频率，已广泛应用于全球汽车产业企业的研发流程中 (上海同星智能) 。作为虚拟仪器软件平台，TSMaster 能够连接和控制所有同星硬件产品，实现车载网络嵌入式代码生成、监控、仿真、开发、诊断、标定、BootLoader、I/O 控制、测试测量以及产线应用（EOL）等全方位功能 (上海同星智能) 。

1.1.1 总线通信支持（CAN/CAN FD/LIN/FlexRay/Ethernet） TSMaster 在总线通信方面提供了业界领先的协议覆盖能力，支持 **CAN、CAN FD、LIN、FlexRay 和 Ethernet** 五种主流车载网络协议 (上海同星智能) 。在 CAN/CAN FD 领域，平台支持 **125Kbps 至 8Mbps 的可调波特率范围**，能够处理标准帧（11 位标识符）和扩展帧（29 位标识符）两种格式，并具备硬件级别的**微秒级时间戳功能**，满足高精度时序分析需求 (tosunai.us) 。CAN FD 模式下，仲裁段和数据段可独立配置波特率，典型组合为仲裁段 500Kbps+ 数据段 2Mbps，兼顾兼容性与传输效率。

LIN 总线支持涵盖 **LIN 1.x/2.x/2.2A 完整协议栈**，提供主节点调度表配置、从节点仿真、诊断帧传输（0x3C/0x3D）以及睡眠/唤醒机制测试等功能 (来源) (CSDN 文库) 。FlexRay 面向底盘控制等确定性实时应用，支持静态段和动态段配置。Ethernet 支持则包括 **100BASE-T1/1000BASE-T1 车载以太网**及 **DoIP（Diagnostic over IP）诊断功能**，满足 ADAS 和自动驾驶系统的大数据量传输需求 (上海同星智能) 。

平台的多协议协同能力尤为突出，**单一工程环境可同时配置和操作多种总线类型**，实现网关 ECU 的跨协议测试。硬件抽象层设计确保代码的硬件无关性，同一套测试脚本可在同星 TC 系列、TSCAN 系列及 Vector、Kvaser、PEAK 等第三方硬件间无缝迁移 (trigopi.com) 。

1.1.2 诊断协议支持（UDS ISO-14229/KWP2000 ISO-14230） TSMaster 的诊断协议支持是其核心竞争力之一。**UDS（Unified Diagnostic Services, ISO 14229）**实现覆盖完整协议栈，包括诊断会话控制（$10）、ECU 复位（$11）、安全访问（$27）、通信控制（$28）、认证服务（$29）、读取数据（$22）、写入数据（$2E）、例程控制（$31）、请求下载（$34）、传输数据（$36）、请求传输退出（$37）等**全部 26 个核心服务** (上海同星智能) 。传输层严格遵循 ISO 15765-2（CAN）和 ISO 17987（LIN），支持单帧（SF）、首帧（FF）、连续帧（CF）、流控帧（FC）的完整传输机制，最长可处理 **4095 字节的多帧数据** (nixinglong.com) 。

KWP2000（Keyword Protocol 2000, ISO 14230）作为 UDS 的前身协议，在 legacy 系统中仍有广泛应用。TSMaster 通过协议适配层支持 KWP2000 的 **K 线/L 线物理层、5 波特慢速初始化、快速初始化机制**，以及服务标识符与 UDS 的映射转换 (CSDN 文库) 。平台允许用户在统一界面中配置两种协议参数，降低了多协议 ECU 的测试复杂度。

诊断数据库管理支持 **CDD（CANdelaStudio）、ODX/PDX、ARXML** 等标准格式导入，自动解析 ECU 支持的服务列表、DID 定义、DTC 描述及 NRC 集合 (CSDN 文库) 。导入后的数据库以树形结构呈现，支持验证功能检查配置一致性。

1.1.3 标定与刷写功能（XCP/CCP/UDS 刷写） TSMaster 集成了专业级的标定与刷写功能模块。**CCP/XCP 标定**支持 A2L 文件加载，兼容 DAQ（Data Acquisition）和 Polling 两种测量模式，采样频率可达 10kHz 以上，支持在线/离线标定、参数曲线调整、MAP 图编辑及标定数据版本管理 (jotactic.com) 。标定数据可导出为 PAR、HEX 等格式，测量数据支持 MDF/MF4 格式记录。

UDS 刷写功能基于 ISO 14229-1 和 ISO 15765-3，实现完整的 BootLoader 流程 (上海同星智能) 。刷写流程分为三阶段：**预编程**（进入扩展会话、解锁安全访问、停止正常通信、关闭 DTC 记录）、**编程**（请求下载 $34→ 传输数据 $36→ 请求传输退出 $37，支持断点续传和错误恢复）、**后编程**（软件完整性校验、ECU 复位、恢复通信）。支持 **S19、HEX、**

BIN、MOT、VDF 等固件格式，提供擦除策略配置（无自动擦除/按 Hex 地址/按数据块）、CRC32/CRC16/自定义校验和算法、以及加密文件处理 (CSDN 博客) 。刷写进度通过系统变量实时反馈，便于集成到自动化流程。

1.1.4 自动化测试与数据分析模块 TSMaster 的自动化测试体系由多层次模块构成。**图形化编程模块**提供无代码测试流程设计，通过拖拽方式编排测试步骤，支持循环、分支、子程序调用等控制结构，内置在线调试器实现单步跟踪和变量监控 (上海同星智能) 。**C/Python 小程序**提供数百个 API 函数，涵盖系统管理、通信控制、数据库访问、诊断服务调用等全功能域，代码具有硬件无关性，便于跨平台复用 (CSDN 博客) 。

测试判定（Verdict）机制是自动化核心，支持 `check_verdict` 等 API 实现信号值范围判定、响应时间判定、诊断结果判定 (上海同星智能) 。判定结果自动记录到测试报告，支持 **PASS/FAIL/INCONCLUSIVE** 三种状态，可配置错误处理策略（继续执行/停止测试/跳转指定步骤）。报告支持 **HTML、PDF、Excel** 等格式导出，包含测试摘要、详细步骤记录、信号曲线截图等 (上海同星智能) 。

数据分析模块提供 **Trace 窗口**（原始报文查看）、**图形窗口**（信号曲线绘制）、**数字窗口**（信号数值显示）及**自定义面板**（用户界面设计）等多种视图 (jotactic.com) 。记录数据支持 **BLF、ASC、CSV、MAT** 等格式，兼容 Vector CANoe、MATLAB 等第三方工具导入。

1.2 开发环境配置

1.2.1 TSMaster 软件安装与硬件接口配置 TSMaster 软件获取途径包括同星官网（www.tosunai.cn）、技术交流群及企业授权分发 (上海同星智能) 。官网提供 **x86（32 位）和 x64（64 位）**两个版本，x64 版本支持更大内存寻址，推荐用于大数据量场景 (上海同星智能) 。安装过程采用向导式界面，支持中英文等六种语言，建议完整安装所有组件以确保开发灵活性 (清华大学出版社) 。

硬件接口配置步骤:

步骤	操作内容	关键参数
1	硬件选择	同星 TC1012/TC1014/TC1018/TC1038-Pro 等，或第三方 Vector/Kvaser/PEAK 设备 (上海同星智能)
2	通道配置	指定 CAN/CAN FD/LIN/Ethernet 通道数量
3	波特率设置	CAN FD：仲裁段 500Kbps，数据段 2Mbps/5Mbps/8Mbps；经典 CAN：125K/250K/500K/1000Kbps (上海同星智能)
4	采样点调整	仲裁段 75%-87.5%，数据段 60%-70%
5	终端电阻	软件配置 120Ω 终端电阻使能
6	通道映射	虚拟通道与物理通道绑定，支持异构硬件统一调度 (上海同星智能)
7	连接验证	发送测试帧，检查总线负载和错误计数器

以 **TC1017** 为例，该 8 通道 CAN FD 转 USB 设备采用 USB 3.0 高速传输，时间戳精度 1 s，CAN 通道支持 **DC 2500V 隔离保护**，内置 120Ω 终端电阻软件可配置 (trigopi.com) 。连接成功后，About 窗口显示已授权 License 列表，确认功能模块可用性 (上海同星智能) 。

1.2.2 COM API 接口启用与权限设置 COM API 基于 Microsoft COM/DCOM 技术，为 Windows 平台提供语言无关的自动化接口。启用步骤:

1. **TSMaster 内部启用**：通过”工具-选项-API”菜单勾选”启用 COM 服务器”(上海同星智能)
2. **DCOM 权限配置**（远程/服务场景）：运行 `dcomcnfg`，导航至”组件服务-计算机-我的电脑-DCOM 配置-TSMaster.Application”，配置启动和访问权限，添加专用服务账户 (上海同星智能)
3. **防火墙配置**：RPC API（基于 gRPC）需开放特定 TCP 端口 (上海同星智能)

ProgID 为 **“TSMaster.Application”**，CLSID 可通过 OLE/COM 对象查看器确认。建议以管理员身份运行 TSMaster 和相关脚本，避免权限不足导致的接口调用失败 (CSDN 博客) 。

1.2.3 Python/C#/C++ 开发环境准备

开发语言	开发工具	关键依赖	推荐版本	适用场景
Python	VS Code/PyCharm	pywin32/comtypes/python-tsmasterapi	Python 3.8+	快速原型、脚本自动化、数据分析
C#	Visual Studio 2019+	.NET Framework 4.5+ / .NET 6+	VS 2019+	企业级应用、复杂 UI、团队协作
C++	VS 2019+/CLion	Windows SDK, TSMaster SDK	C++11+	HIL 仿真、高性能实时应用
LabVIEW	NI LabVIEW 2018+	TSMaster LabVIEW Toolkit	LV 2018+	图形化测试系统、硬件在环

Python 环境验证代码：

```python
import win32com.client
ts = win32com.client.Dispatch("TSMaster.Application")
print(ts.Version)  # 成功返回版本信息即配置正确
```

1.2.4 依赖库安装（pywin32/comtypes/ctypes 替代方案）

方案	安装命令	特点	适用场景
pywin32（首选）	`pip install pywin32==306`	功能完整，IDE 智能提示好	标准 Windows 开发环境
comtypes（轻量替代）	`pip install comtypes==1.4.16`	纯 Python，跨版本兼容好	精简环境、独立部署
python-tsmasterapi（独立模块）	`pip install python-tsmasterapi`	无需 COM，跨平台潜力	云原生、嵌入式 Linux (CSDN 博客)
ctypes（底层调用）	内置标准库	零开销，完全控制	极致性能、特殊平台

若 pywin32 安装失败，可从 Christoph Gohlke 非官方 Windows 二进制包站点下载预编译 whl 文件。对于无法使用 COM 的环境，**libTSCANAPI** 提供 C 风格函数接口，通过 `ctypes` 直接调用 DLL 导出函数 (上海同星智能) 。

1.3 API 接口类型选择

1.3.1 COM API（Windows 平台自动化首选） COM API 是 TSMaster 为 Windows 平台提供的标准自动化接口，采用**双接口（Dual Interface）设计**，支持早期绑定（vtable 调用，高效率）和晚期绑定（IDispatch，灵活性）两种模式 (上海同星智能) 。对象模型层次清晰：**Application→Project→Configuration/Bus/Diagnostic/Measurement**，直观对应 TSMaster 的功能模块划分。

核心优势：与 Windows 生态深度集成，可无缝操作 Excel/Word/数据库生成报告；事件回调机制（`OnMessageReceived`、`OnSignalChanged` 等）支持实时数据监测；与 ECU-TEST、NI TestStand 等第三方测试平台集成成熟（上海同星智能）。**典型延迟 0.1-1ms/调用**，满足大多数自动化测试场景。

局限性：Windows 平台绑定，无法直接用于 Linux；需处理 COM 线程模型（STA/MTA）复杂性。对于标准自动化测试系统，COM API 是**功能完整性和开发效率的最佳平衡点**（上海同星智能）。

1.3.2 Python API（独立模块，无需 COM） **python-tsmasterapi** 以独立 Python 模块形式发布，通过 Cython 封装核心功能，**无需 Windows COM 机制**。支持 Python 3.7-3.11，安装包约 16MB，可从 piwheels 获取预编译版本（piwheels）。

核心模块 `tsapp` 提供硬件无关的 Pythonic 接口：

```
import tsapp
api = tsapp.API()  # 创建实例，自动加载底层库
api.can_send_message(channel=0, id=0x123, data=[0x01, 0x02, 0x03])
```

优势：启动速度快、内存占用低、**跨平台支持（Windows/Linux）**、与现代 Python 生态（NumPy/Pandas/Matplotlib/Pytest）无缝集成（CSDN 博客）。**局限**：部分高级功能（复杂图形界面自动化）可能不如 COM API 完整。推荐用于**快速原型开发、数据分析集成、云原生部署**场景。

1.3.3 C API（高性能实时应用） C API 以 **TSMaster.dll/libTSMaster.so** 形式提供，函数命名遵循 `ts[模块]_[操作]` 规范（如 `tsapp_connect`、`tscan_add_message`、`tsdiag_send_request`）（上海同星智能）。采用句柄模式管理资源，**单次调用延迟微秒级**，支持多线程并发和 CPU 亲和性设置。

典型应用：硬件在环（HIL）仿真微秒级响应、生产线 EOL 测试高速执行、嵌入式目标代码生成。TSMaster 的**嵌入式代码生成功能**基于 C API，将配置的通信/诊断/标定逻辑自动转换为可独立编译的 C 代码（trigopi.com）。开发需处理内存分配、错误码检查、资源释放等底层细节，技术门槛较高。

1.3.4 LabVIEW API（图形化测试系统） LabVIEW API 以 **VI 库（Virtual Instrument Library）**形式提供，封装 TSMaster 功能为图形化节点，支持数据流编程、并行循环、事件结构等 LabVIEW 原生模式（上海同星智能）。适合已有 LabVIEW 测试系统的企业扩展总线/诊断能力，或测试工程师快速搭建原型。**与 NI-CAN/NI-XNET 相比**，优势在于成本效益和协议覆盖度，保护现有技术投资。

2. 总线通信自动化测试实现

2.1 CAN/CAN FD 通信测试

2.1.1 通道配置与波特率设置 CAN 通道配置通过 `Network.Channels` 集合访问，关键参数包括协议类型、双段波特率、采样点、终端电阻。CAN FD 的**仲裁段与数据段独立配置**是核心特性：

```
# Python COM API 通道配置示例
can1 = network.Channels(1)
can1.Protocol = 1  # 0=CAN, 1=CAN FD
can1.ArbitrationBaudrate = 500000      # 仲裁段 500Kbps
can1.DataBaudrate = 2000000            # 数据段 2Mbps
can1.ArbitrationSamplePoint = 80       # 仲裁段采样点 80%
can1.DataSamplePoint = 70              # 数据段采样点 70%
can1.TerminalResistor = True           # 启用终端电阻
can1.Start()
```

波特率选择原则：动力 CAN 典型 500Kbps，舒适 CAN 125Kbps，诊断 CAN 250K/500Kbps；CAN FD 数据段 2Mbps 为性价比平衡点，5Mbps/8Mbps 需评估线缆质量和 EMC 风险。采样点位置影响位定时容错——**长总线或高节点数网络适当提前采样点（75%），短总线高质量信号可延后（87.5%）**（CSDN 博客）。

2.1.2 报文发送与接收（标准帧/扩展帧） TSMaster 支持**标准帧（11 位 ID，0x000-0x7FF）**和**扩展帧（29 位 ID，0x00000000-0x1FFFFFFF）**，以及数据帧、远程帧、错误帧等完整帧类型（上海同星智能）。

发送模式	实现方式	适用场景
单次发送	`Transmit(msg)` 立即执行	诊断请求、事件触发
周期发送	`CyclicMessages.Add(msg, period_ms)`	模拟运行态传感器信号
事件触发	条件判断 +`Transmit()` 调用	响应特定状态变化
序列发送	脚本编排多报文时序	复杂测试场景

接收机制对比：

模式	实现方式	特点
事件驱动	`OnMessageReceived` 回调	响应及时，CPU 占用低，适合高频数据采集
轮询查询	`Receive(timeout)` 定时调用	实现简单，可能引入延迟，适合低频率监测

扩展帧发送示例（UDS 物理寻址）：

```
msg_ext = app.CreateMessage()
msg_ext.ID = 0x18DAF110        # 29 位诊断请求 ID
msg_ext.IsExtended = True
msg_ext.DLC = 8
msg_ext.Data = [0x02, 0x10, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00]  # UDS 10 03
network.Transmit(msg_ext)
```

2.1.3 周期报文与事件触发报文管理 **RBS（Remaining Bus Simulation，残余总线仿真）**模块提供工业级周期报文管理能力。2024 版本重要更新：**“API 添加的周期报文变得更加精确”**（上海同星智能），抖动控制在微秒级别。

周期报文动态管理：

```
# 添加周期报文
rpm_msg = app.CreateMessage()
rpm_msg.ID = 0x100
rpm_msg.DLC = 8
rpm_msg.CycleTime = 10  # 10ms 周期

network.CyclicMessages.Add(rpm_msg)
network.CyclicMessages.StartAll()

# 运行时动态修改
def update_engine_speed(rpm_value):
    # 更新信号值，自动触发报文数据更新
```

```
    network.Signals("EngineData/EngSpeed").Value = rpm_value
    new_msg = network.GetMessageBySignal("EngineData/EngSpeed")
    network.CyclicMessages.UpdateData(rpm_msg.ID, new_msg.Data)

# 停止周期发送
network.CyclicMessages.StopAll()
network.CyclicMessages.Clear()
```

事件触发报文通过系统变量变化、信号阈值跨越、诊断响应到达等事件驱动。TSMaster 的**图形化编程模块**支持流程图式逻辑编排，C/Python 小程序提供 `timer` 对象（1ms 分辨率）实现精确时序控制 (CSDN 博客) 。

2.1.4 信号解析与 DBC 文件加载 DBC 文件加载与信号访问：

```
# 加载 DBC 数据库
db = network.Databases.Add("Powertrain.dbc")
db.Activate = True

# 按名称访问信号（推荐方式）
engine_speed = network.Signals("EngineECU/EngineData/EngSpeed")
engine_speed.Value = 3000  # 设置值，自动编码到关联报文

# 物理值与原始值转换
# 物理值 = 原始值 × Factor + Offset
print(f"当前转速: {engine_speed.Value} {engine_speed.Unit}")
print(f"原始值: {engine_speed.RawValue}, 因子: {engine_speed.Factor}, 偏移: {engine_speed.Offset}")
```

多 DBC 文件管理：同时加载多个数据库，自动处理 ID 冲突（后加载优先或提示合并），支持跨文件信号引用。ARXML（AUTOSAR）格式同样支持导入，满足 AUTOSAR 架构项目需求 (trigopi.com) 。

2.1.5 总线负载分析与错误帧检测 总线负载实时监控与判定：

```
def monitor_bus_load(duration_sec=10, threshold_percent=70):
    """ 监控总线负载，超阈值记录警告"""
    overload_events = []

    for _ in range(int(duration_sec / 0.1)):  # 100ms 采样
        load = can1.BusLoad
        if load > threshold_percent:
            overload_events.append({
                'time': time.time(),
                'load': load
            })
        time.sleep(0.1)

    # 统计报告
    history = can1.BusLoadHistory
    return {
        'average_load': sum(h.Value for h in history) / len(history),
        'max_load': max(h.Value for h in history),
        'overload_count': len(overload_events),
        'overload_events': overload_events
```

```
    }
```

错误帧类型识别：位错误、填充错误、CRC 错误、格式错误、应答错误。TSMaster Trace 窗口以红色高亮显示错误帧，标注错误类型和位置。**错误帧突发通常指示物理层问题**——终端电阻缺失、线缆阻抗异常、节点收发器故障等（上海同星智能）。自动化测试应集成错误帧计数检查，作为测试通过的必要条件。

2.2 LIN 通信测试

2.2.1 LIN 主从节点配置　LIN 采用**单主多从拓扑**，TSMaster 支持主节点（Master）和从节点（Slave）两种模式。主节点配置需定义**调度表（Schedule Table）**——报文传输的时序规划：

```
# LIN 主节点配置
lin_master = network.LINChannels(1)
lin_master.Mode = 0          # 0=Master, 1=Slave
lin_master.Baudrate = 19200  # 典型值 9600 或 19200

# 创建调度表
schedule = lin_master.ScheduleTables.Add("Normal")
schedule.RunMode = 0  # 0=Continuous 循环, 1=Once 单次, 2=Event 事件触发

# 添加调度条目（报文 Protected ID, 延迟 ms）
schedule.Entries.Add(0x10, 10)   # 发动机温度，10ms 后
schedule.Entries.Add(0x11, 20)   # 车速，再 20ms
schedule.Entries.Add(0x12, 15)   # 电池电压，再 15ms

schedule.Start()
```

从节点配置——预定义响应内容，主节点报文头触发后自动发送：

```
lin_slave = network.LINChannels(1)
lin_slave.Mode = 1  # Slave 模式

# 配置对 ID 0x20 的响应
response = lin_slave.ResponseEntries.Add(0x20)
response.DLC = 4
response.Data = [0x12, 0x34, 0x56, 0x78]

# 动态更新响应数据
def update_slave_response(temp, voltage):
    response.Data = [
        temp & 0xFF, (temp >> 8) & 0xFF,      # 温度 2 字节
        voltage & 0xFF, (voltage >> 8) & 0xFF  # 电压 2 字节
    ]
```

2.2.2 调度表管理与诊断帧传输　**多调度表动态切换**适应不同工作模式：

```
# 创建诊断专用调度表
diag_schedule = lin_master.ScheduleTables.Add("Diagnostic")
diag_schedule.Entries.Add(0x3C, 50)  # 诊断请求
diag_schedule.Entries.Add(0x3D, 0)   # 诊断响应期待

# 模式切换
```

```
normal_schedule.Stop()
diag_schedule.Start()

# 发送 UDS on LIN 诊断请求
diag_req = app.CreateLINMessage()
diag_req.ID = 0x3C
diag_req.Data = [0x02, 0x10, 0x03] + [0xFF]*5  # UDS 10 03
lin_master.Transmit(diag_req)
```

LIN 诊断基于 **ISO 17987**，使用 0x3C（主请求）和 0x3D（从响应）进行单帧/多帧传输。TSMaster 传输层自动处理首帧-连续帧-流控帧交互，应用层关注服务数据内容 (CSDN 文库) 。

2.2.3 LIN 睡眠/唤醒机制测试

睡眠/唤醒是车身电子低功耗验证的关键：

```
def test_lin_sleep_wake():
    # 发送睡眠命令
    sleep_cmd = app.CreateLINMessage()
    sleep_cmd.ID = 0x3C
    sleep_cmd.Data = [0x00] + [0xFF]*7  # 第一字节 0x00 表示睡眠
    lin_master.Transmit(sleep_cmd)

    # 验证总线静默（<4 秒自动进入睡眠，或立即响应睡眠命令）
    time.sleep(0.5)
    assert check_bus_activity() == 0, "总线未进入静默"

    # 发送唤醒信号（显性电平 150μs-5ms）
    lin_master.SendWakeUpPulse(duration_ms=2)

    # 测量唤醒响应时间（典型 <100ms）
    wake_start = time.time()
    while time.time() - wake_start < 0.5:
        msg = lin_master.Receive(timeout=0.01)
        if msg and msg.ID == 0x80:  # 唤醒信号
            return (time.time() - wake_start) * 1000  # ms

    raise TimeoutError("唤醒响应超时")
```

2.3 通信测试脚本实例

2.3.1 Python COM API 报文收发示例

完整测试框架：

```
#!/usr/bin/env python3
"""
TSMaster CAN 通信自动化测试示例
功能：多通道配置、周期报文发送、事件接收、质量监控
"""

import win32com.client
import time
import json
from datetime import datetime
```

```python
from dataclasses import dataclass, asdict

@dataclass
class TestConfig:
    channel_count: int = 2
    arbitration_baudrate: int = 500000
    data_baudrate: int = 2000000
    test_duration_sec: float = 30.0
    dbc_path: str = "Powertrain.dbc"

class TSMasterCANTest:
    def __init__(self):
        self.app = None
        self.network = None
        self.channels = {}
        self.received_messages = []
        self.test_log = []

    def initialize(self) -> bool:
        """ 初始化 TSMaster 连接和配置"""
        try:
            self.app = win32com.client.Dispatch("TSMaster.Application")
            self.log(f"TSMaster 版本: {self.app.Version}")

            # 创建新项目或打开现有项目
            if self.app.CurrentProject is None:
                self.app.NewProject()

            self.network = self.app.CurrentProject.Network

            # 配置通道
            self._configure_channels()

            # 加载 DBC
            if self.config.dbc_path:
                self._load_database()

            # 设置接收回调
            self._setup_receivers()

            return True

        except Exception as e:
            self.log(f"初始化失败: {e}", level="ERROR")
            return False

    def _configure_channels(self):
        """ 配置 CAN 通道参数"""
        for i in range(self.config.channel_count):
            ch = self.network.Channels(i + 1)
            ch.Protocol = 1  # CAN FD
```

```
            ch.ArbitrationBaudrate = self.config.arbitration_baudrate
            ch.DataBaudrate = self.config.data_baudrate
            ch.ArbitrationSamplePoint = 80
            ch.DataSamplePoint = 70
            ch.TerminalResistor = True
            ch.Start()

            self.channels[i+1] = {
                'object': ch,
                'messages_sent': 0,
                'messages_received': 0,
                'error_count': 0
            }
            self.log(f"通道{i+1}启动: {ch.ArbitrationBaudrate/1000:.0f}K/{ch.DataBaudrate/1000000:.0f}M")

    def _load_database(self):
        """ 加载 DBC 数据库"""
        db = self.network.Databases.Add(self.config.dbc_path)
        db.Activate = True
        self.log(f"数据库加载: {self.config.dbc_path}")

    def _setup_receivers(self):
        """ 设置报文接收处理"""
        # 使用 WithEvents 注册回调（需定义事件处理类）
        for ch_id, ch_info in self.channels.items():
            handler = self._create_receiver_handler(ch_id)
            ch_info['receiver'] = win32com.client.WithEvents(
                ch_info['object'], handler
            )

    def _create_receiver_handler(self, channel_id):
        """ 创建通道接收事件处理类"""
        class ReceiverHandler:
            def __init__(self, outer, ch_id):
                self.outer = outer
                self.ch_id = ch_id

            def OnMessageReceived(self, msg):
                event = {
                    'channel': self.ch_id,
                    'timestamp': time.time(),
                    'id': hex(msg.ID),
                    'dlc': msg.DLC,
                    'data': list(msg.Data[:msg.DLC]),
                    'is_extended': msg.IsExtended,
                    'is_fd': msg.IsFD,
                    'brs': msg.BRS
                }
                self.outer.received_messages.append(event)
                self.outer.channels[self.ch_id]['messages_received'] += 1
```

```
            # 实时信号解析（如 DBC 已加载）
            try:
                signal_values = self.outer._parse_signals(msg)
                event['signals'] = signal_values
            except:
                pass

    return lambda outer=self, ch_id=channel_id: ReceiverHandler(outer, ch_id)

def send_test_sequence(self):
    """ 发送测试报文序列"""
    # 创建周期报文
    engine_msg = self.app.CreateMessage()
    engine_msg.ID = 0x100
    engine_msg.IsFD = True
    engine_msg.BRS = True
    engine_msg.DLC = 64  # CAN FD 最大 64 字节

    # 启动 10ms 周期发送
    self.network.CyclicMessages.Add(engine_msg, 10)
    self.network.CyclicMessages.StartAll()

    # 动态数据更新循环
    start_time = time.time()
    while time.time() - start_time < self.config.test_duration_sec:
        # 模拟发动机转速变化
        elapsed = time.time() - start_time
        rpm = 1000 + int(4000 * (elapsed / self.config.test_duration_sec))
        rpm = min(rpm, 5000)

        # 更新信号值（假设 DBC 已配置）
        try:
            self.network.Signals("EngineData/EngSpeed").Value = rpm
            self.network.Signals("EngineData/EngTemp").Value = 80 + int(20 * (rpm / 5000))
            updated = self.network.GetMessageBySignal("EngineData/EngSpeed")
            self.network.CyclicMessages.UpdateData(0x100, updated.Data)
        except:
            # 无 DBC 时直接操作数据字节
            pass

        # 周期性状态记录
        if int(elapsed) % 5 == 0:  # 每 5 秒
            self._record_status()

        time.sleep(0.05)

    self.network.CyclicMessages.StopAll()

def _record_status(self):
    """ 记录通道状态"""
    for ch_id, ch_info in self.channels.items():
```

```python
        ch = ch_info['object']
        status = {
            'timestamp': time.time(),
            'channel': ch_id,
            'bus_load': ch.BusLoad,
            'tx_frames': ch.TransmitFrameCount,
            'rx_frames': ch.ReceiveFrameCount,
            'error_frames': ch.ErrorFrameCount,
            'status': ch.Status
        }
        self.test_log.append(status)

    def generate_report(self) -> dict:
        """ 生成测试报告"""
        # 消息统计
        msg_by_channel = {}
        for msg in self.received_messages:
            ch = msg['channel']
            msg_by_channel[ch] = msg_by_channel.get(ch, 0) + 1

        # 唯一 ID 统计
        unique_ids = set(m['id'] for m in self.received_messages)

        report = {
            'test_time': datetime.now().isoformat(),
            'configuration': asdict(self.config),
            'duration_sec': self.config.test_duration_sec,
            'messages_sent_estimate': int(self.config.test_duration_sec * 100),  # 10ms 周期
            'messages_received': len(self.received_messages),
            'messages_by_channel': msg_by_channel,
            'unique_ids': list(unique_ids),
            'final_bus_load': {ch_id: ch['object'].BusLoad
                               for ch_id, ch in self.channels.items()},
            'final_error_count': {ch_id: ch['object'].ErrorFrameCount
                                  for ch_id, ch in self.channels.items()}
        }

        # 保存详细日志
        with open(f"can_test_{int(time.time())}.json", 'w') as f:
            json.dump({
                'summary': report,
                'messages': self.received_messages[-1000:],  # 最近 1000 条
                'status_log': self.test_log
            }, f, indent=2, default=str)

        return report

    def log(self, message: str, level: str = "INFO"):
        """ 记录日志"""
        entry = f"[{datetime.now().strftime('%H:%M:%S.%f')[:-3]}] [{level}] {message}"
        print(entry)
```

```
        self.test_log.append({'type': 'log', 'content': entry})

    def cleanup(self):
        """ 清理资源"""
        for ch_info in self.channels.values():
            try:
                ch_info['object'].Stop()
            except:
                pass
        if self.app:
            self.app.Quit()

# 执行入口
if __name__ == '__main__':
    config = TestConfig(
        channel_count=2,
        test_duration_sec=10.0,
        dbc_path="Demo/Powertrain.dbc"
    )

    test = TSMasterCANTest()
    test.config = config

    try:
        if test.initialize():
            test.send_test_sequence()
            report = test.generate_report()
            print(f"\n测试完成: 接收{report['messages_received']}条消息")
            print(f"总线负载: {report['final_bus_load']}")
        else:
            print("初始化失败，测试中止")
    finally:
        test.cleanup()
```

2.3.2 信号值判定与自动化 verdict 机制 基于 TSMaster `check_verdict` API 的判定框架:

```
class SignalVerifier:
    def __init__(self, network):
        self.network = network
        self.verdicts = []

    def check_range(self, signal_path: str, min_val: float, max_val: float,
                    description: str, duration_ms: int = 0) -> dict:
        """
        信号范围判定
        duration_ms: 0 表示即时判定，>0 表示时段判定
        """
        if duration_ms == 0:
            # 即时判定
            signal = self.network.Signals(signal_path)
            actual = signal.Value
```

```
        passed = min_val <= actual <= max_val

        verdict = {
            'type': 'instant',
            'description': description,
            'signal': signal_path,
            'expected': f'[{min_val}, {max_val}]',
            'actual': actual,
            'unit': signal.Unit,
            'result': 'PASS' if passed else 'FAIL',
            'timestamp': time.time()
        }
    else:
        # 时段判定：监测整个时间窗口内的信号行为
        samples = []
        start = time.time()
        while (time.time() - start) * 1000 < duration_ms:
            samples.append(self.network.Signals(signal_path).Value)
            time.sleep(0.01)

        actual_min, actual_max = min(samples), max(samples)
        passed = min_val <= actual_min and actual_max <= max_val

        verdict = {
            'type': 'time_window',
            'description': description,
            'signal': signal_path,
            'expected_range': f'[{min_val}, {max_val}]',
            'actual_range': f'[{actual_min:.3f}, {actual_max:.3f}]',
            'samples': len(samples),
            'always_in_range': passed,
            'result': 'PASS' if passed else 'FAIL'
        }

    self.verdicts.append(verdict)

    # 同步到系统变量供其他模块使用
    sv_name = f"Verdict_{signal_path.replace('/', '_')}"
    self._ensure_system_var(sv_name, 1 if passed else 0)

    return verdict

def check_settling_time(self, signal_path: str, target: float,
                        tolerance: float, timeout_ms: int,
                        description: str) -> dict:
    """ 检查信号稳定到目标值的时间"""
    signal = self.network.Signals(signal_path)
    start = time.time()

    while (time.time() - start) * 1000 < timeout_ms:
        current = signal.Value
```

```
        if abs(current - target) <= tolerance:
            # 进入稳定期验证
            stable_start = time.time()
            while (time.time() - stable_start) * 1000 < 100:  # 100ms 稳定期
                if abs(signal.Value - target) > tolerance:
                    break  # 不稳定，重新开始
                time.sleep(0.01)
            else:
                # 稳定期通过
                return {
                    'description': description,
                    'signal': signal_path,
                    'target': target,
                    'tolerance': tolerance,
                    'settle_time_ms': (time.time() - start) * 1000,
                    'result': 'PASS'
                }
        time.sleep(0.005)

    # 超时失败
    return {
        'description': description,
        'signal': signal_path,
        'target': target,
        'final_value': signal.Value,
        'timeout_ms': timeout_ms,
        'result': 'FAIL'
    }

def get_overall_verdict(self) -> str:
    """ 获取综合测试结论"""
    if not self.verdicts:
        return 'INCONCLUSIVE'
    if any(v['result'] == 'FAIL' for v in self.verdicts):
        return 'FAIL'
    if all(v['result'] == 'PASS' for v in self.verdicts):
        return 'PASS'
    return 'INCONCLUSIVE'

def generate_report(self) -> dict:
    """ 生成详细判定报告"""
    return {
        'overall_verdict': self.get_overall_verdict(),
        'total_checks': len(self.verdicts),
        'pass_count': sum(1 for v in self.verdicts if v['result'] == 'PASS'),
        'fail_count': sum(1 for v in self.verdicts if v['result'] == 'FAIL'),
        'details': self.verdicts
    }

def _ensure_system_var(self, name: str, value):
    """ 确保系统变量存在并设置值"""
```

```
        try:
            sv = self.network.SystemVars(name)
        except:
            sv = self.network.SystemVars.Add(name)
        sv.Value = value
```

2.3.3 多通道同步测试实现 多通道精确同步测试框架:

```
import threading
import time

class SynchronizedMultiChannelTest:
    def __init__(self, app):
        self.app = app
        self.network = app.CurrentProject.Network
        self.channels = {}
        self.sync_barrier = threading.Barrier(1)  # 动态调整
        self.global_timebase = None

    def initialize_channels(self, configs: list):
        """ 初始化多通道，配置统一时间基准"""
        self.sync_barrier = threading.Barrier(len(configs))

        for cfg in configs:
            ch_id = cfg['id']
            ch = self.network.Channels(ch_id)

            # 应用配置
            ch.Protocol = cfg.get('protocol', 1)  # CAN FD 默认
            ch.ArbitrationBaudrate = cfg['arbitration_baudrate']
            ch.DataBaudrate = cfg.get('data_baudrate', cfg['arbitration_baudrate'])
            ch.TerminalResistor = cfg.get('terminal', True)

            # 关键: 启用硬件时间戳同步
            ch.HardwareTimestampEnabled = True
            ch.Start()

            self.channels[ch_id] = {
                'object': ch,
                'config': cfg,
                'thread': None,
                'results': []
            }

        # 校准全局时间基准
        self._calibrate_timebase()

    def _calibrate_timebase(self):
        """ 校准各通道的硬件时间基准"""
        # 发送同步报文，测量往返延迟
        sync_times = {}
```

```python
        for ch_id, ch_info in self.channels.items():
            ch = ch_info['object']
            # 获取当前硬件时间
            sync_times[ch_id] = ch.GetHardwareTimestamp()

        # 计算相对偏移，建立统一时间轴
        self.global_timebase = min(sync_times.values())
        self.channel_offsets = {
            ch_id: ts - self.global_timebase
            for ch_id, ts in sync_times.items()
        }

    def execute_synchronized_action(self, action_func, sync_delay_us: int = 0):
        """ 在所有通道上执行精确同步动作"""
        results = {}
        threads = []

        def channel_worker(ch_id, ch_info):
            # 等待同步屏障
            self.sync_barrier.wait()

            # 精确延时到目标时刻
            if sync_delay_us > 0:
                target_time = time.perf_counter() + sync_delay_us / 1e6
                while time.perf_counter() < target_time:
                    pass  # 忙等待确保微秒级精度

            # 执行动作
            result = action_func(ch_info['object'])
            results[ch_id] = {
                'result': result,
                'exec_time': time.perf_counter(),
                'hw_timestamp': ch_info['object'].GetHardwareTimestamp()
            }

        # 启动各通道工作线程
        for ch_id, ch_info in self.channels.items():
            t = threading.Thread(target=channel_worker, args=(ch_id, ch_info))
            t.start()
            threads.append(t)

        # 等待全部完成
        for t in threads:
            t.join(timeout=5.0)

        return results

    def send_synchronized_messages(self, channel_messages: dict,
                                   sync_accuracy_us: int = 100):
        """
        多通道同步发送报文
```

```python
    channel_messages: {channel_id: {'id': int, 'data': list, 'is_fd': bool}}
    """
    def send_action(ch, msg_def):
        msg = self.app.CreateMessage()
        msg.ID = msg_def['id']
        msg.DLC = len(msg_def['data'])
        msg.Data = msg_def['data']
        msg.IsFD = msg_def.get('is_fd', True)
        msg.BRS = msg_def.get('brs', True)
        ch.Transmit(msg)
        return {'sent': True, 'id': msg_def['id']}

    # 准备各通道的动作函数
    actions = {}
    for ch_id, msg_def in channel_messages.items():
        actions[ch_id] = lambda ch, md=msg_def: send_action(ch, md)

    # 统一同步执行
    return self.execute_synchronized_action(
        lambda ch, ch_id=ch_id: actions.get(ch_id, lambda x: None)(ch),
        sync_delay_us=sync_accuracy_us
    )

def verify_cross_channel_timing(self, tx_channel: int, rx_channel: int,
                                expected_delay_ms: float, tolerance_ms: float):
    """ 验证跨通道报文传输时序"""
    # 在发送通道发送带时间戳的测试报文
    tx_time = time.perf_counter()
    test_msg = self.app.CreateMessage()
    test_msg.ID = 0x7FF  # 测试专用 ID
    test_msg.Data = list(struct.pack('>d', tx_time))  # 嵌入发送时间戳
    self.channels[tx_channel]['object'].Transmit(test_msg)

    # 在接收通道等待并解析
    rx_ch = self.channels[rx_channel]['object']
    timeout = expected_delay_ms + tolerance_ms + 10  # 额外余量

    start_wait = time.perf_counter()
    while (time.perf_counter() - start_wait) * 1000 < timeout:
        msg = rx_ch.Receive(timeout=0.001)
        if msg and msg.ID == 0x7FF:
            rx_time = time.perf_counter()
            embedded_tx = struct.unpack('>d', bytes(msg.Data[:8]))[0]

            measured_delay = (rx_time - embedded_tx) * 1000  # ms
            routing_delay = measured_delay - (embedded_tx - tx_time) * 1000

            return {
                'measured_total_ms': measured_delay,
                'routing_delay_ms': routing_delay,
```

```python
                    'within_tolerance': abs(routing_delay - expected_delay_ms) <= tolerance_ms
                }

        return {'error': 'timeout', 'expected_delay_ms': expected_delay_ms}

# 网关路由延迟测试示例
def test_gateway_routing():
    """ 测试 CAN1 到 CAN2 的网关路由延迟"""
    app = win32com.client.Dispatch("TSMaster.Application")
    test = SynchronizedMultiChannelTest(app)

    test.initialize_channels([
        {'id': 1, 'arbitration_baudrate': 500000, 'data_baudrate': 2000000},
        {'id': 2, 'arbitration_baudrate': 500000, 'data_baudrate': 2000000},
    ])

    # 同步发送测试报文
    results = test.send_synchronized_messages({
        1: {'id': 0x123, 'data': [0xAA, 0xBB, 0xCC, 0xDD, 0x00, 0x11, 0x22, 0x33]}
    })

    # 验证路由延迟
    timing = test.verify_cross_channel_timing(1, 2, expected_delay_ms=5.0, tolerance_ms=2.0)

    print(f"路由延迟: {timing.get('routing_delay_ms', 'N/A'):.3f} ms")
    print(f"判定: {'PASS' if timing.get('within_tolerance') else 'FAIL'}")
```

3. UDS 诊断协议自动化测试

3.1 UDS 诊断模块配置

3.1.1 传输层参数设置（CAN ID/超时/填充字节） UDS 诊断传输层配置决定通信的可靠性和效率。TSMaster 诊断配置界面和 COM API 提供完整参数控制:

参数类别	参数名称	典型值	功能说明
寻址参数	请求 ID 类型	0（标准帧）/1（扩展帧）	诊断请求报文格式
	请求 ID (ReqID)	0x7E0/0x7DF	物理寻址/功能寻址
	响应 ID (ResID)	0x7E8	ECU 响应标识符
	功能 ID (FunctionalID)	0x7DF	广播诊断请求
时序参数	P2 超时	50ms	默认会话响应超时
	P2* 扩展超时	5000ms	收到 0x78 后的扩展时间
	S3 服务端时间	5000ms	非默认会话超时
	S3 客户端时间	2000ms	TesterPresent 发送周期
传输参数	填充字节 (FillByte)	0xAA/0x55/0x00	未使用字节填充
	是否填充	True/False	数据填充使能
	Stmin	0-127ms	连续帧最小间隔

Python COM API 配置示例:

```
diag = app.GetDiagnostic()
config = diag.GetConfig("ECU1_Diag")

# CAN ID 配置
config.RequestID = 0x7E0
config.ResponseID = 0x7E8
config.FunctionalID = 0x7DF
config.RequestIDType = 0  # 标准帧

# 超时参数（单位：ms）
config.P2Timeout = 50
config.P2StarTimeout = 5000
config.S3Timeout = 5000

# 传输层选项
config.FillByte = 0xAA
config.TxPadding = True

config.Apply()
```

关键注意事项：P2 设置过短导致正常响应超时误判，过长延迟错误检测；填充字节需与 ECU 规范严格一致，不匹配可能导致请求被拒绝；CAN FD 场景需额外配置**数据段波特率和可变波特率使能**。

3.1.2 诊断服务预配置（$10/$22/$2E/$34/$35/$36/$37 等） TSMaster 支持将常用 UDS 服务预配置为可复用模板，简化自动化脚本开发：

服务 SID	服务名称	预配置内容	典型应用场景
$10	DiagnosticSessionControl	会话类型（01/02/03/04）、P2 参数	会话切换序列
$11	ECUReset	复位类型（01 硬复位/02 钥匙关/03 软复位/04 使能快速关闭）	故障恢复测试
$22	ReadDataByIdentifier	常用 DID 列表（F190 VIN、F194 软件版本、F193 硬件版本等）	标识信息读取
$2E	WriteDataByIdentifier	可写 DID、数据格式、安全级别要求	参数配置标定
$27	SecurityAccess	安全等级、Seed 长度、Key 长度、算法引用	安全解锁流程
$31	RoutineControl	例程 ID、启动参数、预期结果	内置测试执行
$34	RequestDownload	内存地址格式、数据格式标识符、压缩/加密支持	Bootloader 启动
$36	TransferData	块大小、校验和类型、序列计数器管理	固件数据传输
$37	RequestTransferExit	校验和验证、完整性检查	传输完成确认

预配置服务与 ODX/PDX 数据库导入密切相关。加载标准诊断数据库后，TSMaster 自动生成对应服务配置，保持与

ECU 规范一致性 (CSDN 文库) 。复合服务机制将 $34/$36/$37 集成为单一"DownloadFile" 步骤，自动处理文件解析、地址计算、分段传输 (上海同星智能) 。

3.1.3 Seed&Key 安全算法集成 安全访问（$27）是 UDS 诊断的关键保护机制。TSMaster 2024.03 版本起支持 **C/C++、C#、VB.NET、Delphi** 四种 DLL 接口 (上海同星智能) ：

实现方式	开发工具	接口规范	适用场景
C/C++ DLL	Visual Studio	`GenerateKeyEx` / `GenerateKeyExOpt`	高性能、算法保密
C# .NET DLL	VS 2019+	`IGenerateKeyEx` / `IGenerateKeyExOpt` / `IGenerateKeyExS`	快速开发、.NET 生态
VB.NET DLL	VS 2019+	同上	VB 遗留系统迁移
Delphi DLL	Delphi	传统接口	历史项目维护
内置算法编辑器	TSMaster 内置	类 C 语法	简单算法、快速验证

C# 算法 DLL 开发模板核心:

```
// 实现 IGenerateKeyEx 接口
public class SeedKeyAlgorithm : IGenerateKeyEx
{
    public int GenerateKeyEx(
        byte[] SeedArray,        // 输入: ECU 返回的 Seed
        uint SeedArraySize,
        byte[] KeyArray,         // 输出: 计算的 Key
        uint KeyArraySize,
        uint SecurityLevel,      // 安全等级 (1, 3, 5, 7 等)
        string Variant,          // 变体标识
        string ECUName           // ECU 名称
    )
    {
        // 实现具体算法: 如 AES、RSA、自定义滚动码等
        // 注意: SeedArray 和 KeyArray 为字节数组，需处理大小端

        byte[] key = ComputeKey(SeedArray, (int)SecurityLevel);
        Array.Copy(key, KeyArray, Math.Min(key.Length, KeyArraySize));

        return 0; // 成功返回 0
    }

    private byte[] ComputeKey(byte[] seed, int level)
    {
        // 具体算法实现，受保密协议保护
        // 示例: 简单 XOR (实际项目需使用安全算法)
        byte[] key = new byte[seed.Length];
        byte mask = (byte)(0x5A + level * 0x11);
        for (int i = 0; i < seed.Length; i++)
        {
            key[i] = (byte)(seed[i] ^ mask);
```

```
        }
        return key;
    }
}
```

TSMaster 提供 **DLL 校验器工具**，无需连接 ECU 即可验证接口正确性和算法输出（上海同星智能）。安全算法部署时建议采用**代码混淆、加密壳、硬件狗**等保护措施。

3.1.4 自动化诊断流程（UDS Flow）设计 TSMaster 的 **UDS Flow** 功能通过图形化编辑器编排多步骤诊断序列，支持六种步骤类型（上海同星智能）：

步骤类型	功能描述	典型应用
Normal	直接指定请求和期望响应数据	简单服务测试
SelectConfig	引用诊断控制台预配置服务	复用验证的服务定义
SeedAndKey	安全访问解锁专用步骤	自动 $27 序列
DownloadFile	文件下载专用步骤	Bootloader 刷写
TesterPresent	诊断仪在线保持控制	会话维持管理
RoutineControl	例程控制专用步骤	执行 ECU 内置测试

流程设计最佳实践：

- **模块化组织**：相关步骤分组为子流程，提高可读性
- **参数化配置**：系统变量替代硬编码，增强适应性
- **完善错误处理**：超时、重试、跳转策略确保异常可控
- **结果可判定**：每个步骤关联 PASS/FAIL 判定

流程执行状态通过系统变量实时反馈：`UDSProgress`（整体进度百分比）、`UDSStepResult`（当前步骤结果）、`UDSFlowStatus`（流程状态机）。支持单步调试，便于开发验证（上海同星智能）。

3.2 诊断响应处理与 NRC 解析

3.2.1 肯定响应与否定响应识别机制 UDS 响应识别基于协议规定的格式规范：

响应类型	格式	识别特征	数据提取
肯定响应	[SID+0x40] [子功能] [响应数据]	首字节 = 请求 SID + 0x40	跳过 2 字节头部，剩余为数据
否定响应	0x7F [请求 SID] [NRC]	首字节 = 0x7F	NRC 为第 3 字节
响应挂起	0x7F [请求 SID] 0x78	NRC = 0x78	表示 ECU 处理中，需等待最终响应

TSMaster 诊断引擎自动识别响应类型，通过 COM API 提供结构化访问：

```
response = diag.ExecuteService(request)

if response.IsPositive:
    # 肯定响应处理
    service_data = response.Data[2:]  # 跳过 SID+ 子功能
```

```
    parse_service_data(service_data)
elif response.IsNegative:
    # 否定响应处理
    nrc = response.NRC
    requested_sid = response.RequestedSID
    handle_nrc(nrc, requested_sid)
elif response.IsPending:
    # 0x78 响应挂起，延长等待
    wait_for_final_response(extended_timeout)
```

3.2.2 否定响应码（NRC）定义与分类

3.2.2.1 通用 NRC（0x10-0x13, 0x21-0x24, 0x31-0x33 等）

NRC	名称	触发条件	自动化处理策略
0x10	**GeneralReject**	未分类的服务执行失败	记录详细上下文，人工分析
0x11	**ServiceNotSupported**	请求 SID 未实现	检查 ECU 诊断规范版本
0x12	**SubFunctionNotSupported**	子功能参数无效	验证子功能支持矩阵
0x13	**IncorrectMessageLengthOrInvalidFormat**	长度/格式错误	检查请求数据打包逻辑
0x14	**ResponseTooLong**	响应超传输层容量	启用分段传输或缩小请求
0x21	**BusyRepeatRequest**	ECU 正处理其他请求	**自动延迟重试**（推荐 1-5 秒）
0x22	**ConditionsNotCorrect**	前提条件不满足	检查会话状态、安全级别、车辆状态
0x23	**RoutineNotComplete**	例程尚未完成	延长等待或查询例程状态
0x24	**RequestSequenceError**	请求顺序错误	验证状态机转换合法性
0x31	**RequestOutOfRange**	参数超出有效范围	**边界值测试重点**，验证 DID/RID/地址
0x32	**SecurityAccessDenied**	安全访问被拒绝	检查前置解锁步骤
0x33	**InvalidKey**	密钥无效	验证 Seed&Key 算法正确性

3.2.2.2 安全访问相关 NRC（0x24, 0x35, 0x36, 0x37）

NRC	名称	安全机制	测试关注点
0x24	**RequestSequenceError**	未请求 Seed 直接发送 Key	验证状态机严格性
0x35	**InvalidKey**	密钥计算错误	**暴力破解防护测试**
0x36	**ExceedNumberOfAttempts**	尝试次数超限锁定	验证锁定阈值和恢复条件
0x37	**RequiredTimeDelayNotExpired**	强制延迟期未结束	验证时序安全机制

安全访问测试需系统覆盖：正确解锁流程、错误 Key 拒绝、连续失败锁定、延迟机制、以及跨会话/跨复位的行为一致性。

3.2.2.3 会话控制 NRC（0x7E, 0x7F）

NRC	名称	触发场景	处理策略
0x7E	**SubFunctionNotSupportedInActiveSession**	当前会话不支持子功能	**自动会话切换**后重试
0x7F	**ServiceNotSupportedInActiveSession**	当前会话不支持整个服务	检查服务-会话映射矩阵

会话依赖测试应验证：各服务在不同会话下的可用性、会话切换的响应正确性、超时回退行为、以及嵌套会话请求的处理。

3.2.3 系统变量绑定获取诊断响应 系统变量机制将诊断响应异步解耦，简化脚本开发：

```
# 配置阶段：绑定诊断响应到系统变量
diag_config = diag.GetConfig("ECU1")
diag_config.BindResponseToSystemVar("DiagResponseData", "ResponseData")  # 字节数组
diag_config.BindResponseToSystemVar("DiagResponseCode", "ResponseCode")  # NRC 值
diag_config.BindResponseToSystemVar("DiagResponseValid", "ResponseValid") # 完成标志

# 执行阶段：触发服务并等待响应
sys_vars = app.SystemVariables
sys_vars.Set("DiagRequestTrigger", 1)  # 触发服务执行

# 等待完成（轮询或事件）
while sys_vars.Get("DiagResponseValid") == 0:
    time.sleep(0.001)  # 1ms 轮询

# 读取结果
nrc = sys_vars.Get("DiagResponseCode")
if nrc == 0:
    response_data = sys_vars.Get("DiagResponseData")  # 字节数组
    parse_positive_response(response_data)
else:
    handle_negative_response(nrc)
```

数组类型变量选择：`Uint8 Array` 支持变长诊断数据，但需后续转换；`String` 适合文本化显示；固定长度 `Uint32` 仅适用于已知短响应。推荐**动态数组类型**以适配多帧传输。

3.2.4 响应超时与重试机制处理 分层超时管理策略：

层级	超时类型	典型值	处理动作
传输层	N_Cr（连续帧等待）	1000ms	自动流控帧重发
	N_Bs（流控帧等待）	1000ms	传输终止，上报错误
应用层-P2	默认响应超时	50ms	首次超时检测
应用层-P2*	扩展响应超时	5000ms	收到 0x78 后的等待
应用层-S3	会话保持超时	5000ms	自动 TesterPresent 或会话回退

智能重试策略实现：

```
def execute_with_retry(diag, request, max_retries=3,
                       p2_timeout=50, p2star_timeout=5000):
    """ 带智能重试的诊断服务执行"""

    for attempt in range(max_retries):
        start_time = time.time()

        # 首次尝试使用 P2 超时
        response = diag.ExecuteService(request, timeout=p2_timeout)
```

```
        if response.IsPositive:
            return {'success': True, 'data': response.Data, 'attempts': attempt+1}

        if response.IsPending:  # 0x78
            # 切换到 P2* 扩展超时
            response = diag.WaitForFinalResponse(timeout=p2star_timeout)
            if response.IsPositive:
                return {'success': True, 'data': response.Data, 'attempts': attempt+1}

        # 否定响应分析
        nrc = response.NRC

        if nrc == 0x21:  # Busy，可重试
            delay = min(2 ** attempt, 10)  # 指数退避，最大 10 秒
            time.sleep(delay)
            continue  # 下一次重试

        elif nrc == 0x78:  # 已在上面处理，不应到达
            continue

        elif nrc in [0x22, 0x24]:  # 条件/序列错误，可能需要前置步骤
            return {'success': False, 'error': f'Precondition failed: 0x{nrc:02X}',
                    'recoverable': True}

        else:  # 其他 NRC，通常不可重试
            return {'success': False, 'error': f'NRC: 0x{nrc:02X}',
                    'recoverable': False}

    # 重试耗尽
    return {'success': False, 'error': 'Max retries exceeded', 'recoverable': False}
```

3.3 典型诊断服务测试实现

3.3.1 诊断会话控制（$10 Default/Extended/Programming）

会话控制测试覆盖状态机完整转换：

```
class SessionControlTest:
    def __init__(self, diag):
        self.diag = diag
        self.session_states = ['Default', 'Extended', 'Programming']

    def test_session_transitions(self):
        """ 测试所有合法和非法会话转换"""
        results = []

        # 状态转换矩阵测试
        transitions = [
            # (from, to, expected_result, description)
            ('Default', 'Extended', 'PASS', '正常进入扩展会话'),
            ('Default', 'Programming', 'FAIL', '默认会话不能直接进编程'),
            ('Extended', 'Programming', 'PASS', '扩展会话进编程'),
```

```
        ('Extended', 'Extended', 'PASS', '重复进入扩展会话'),
        ('Programming', 'Default', 'PASS', '编程会话返回默认'),
        ('Programming', 'Extended', 'PASS', '编程会话转扩展'),
    ]

    for from_sess, to_sess, expected, desc in transitions:
        # 建立起始状态
        self._enter_session(from_sess)

        # 尝试转换
        result = self._enter_session(to_sess)
        actual = 'PASS' if result['success'] else 'FAIL'

        # 特殊处理：某些转换预期失败
        if expected == 'FAIL' and not result['success']:
            actual = 'PASS'  # 预期失败，实际失败 = 测试通过

        results.append({
            'transition': f'{from_sess}->{to_sess}',
            'description': desc,
            'expected': expected,
            'actual': actual,
            'nrc': result.get('nrc', 'N/A')
        })

    # 超时测试
    results.append(self._test_session_timeout())

    return results

def _enter_session(self, session_type):
    """ 进入指定会话"""
    session_codes = {'Default': 0x01, 'Extended': 0x03, 'Programming': 0x02}
    request = [0x10, session_codes[session_type]]
    return self.diag.ExecuteService(request)

def _test_session_timeout(self):
    """ 测试 S3 超时回退"""
    # 进入扩展会话
    self._enter_session('Extended')

    # 停止发送 TesterPresent，等待超时
    time.sleep(6)  # S3 典型 5 秒，留余量

    # 验证已回退到默认会话（尝试执行扩展会话专属服务）
    result = self.diag.ExecuteService([0x10, 0x03])  # 再次进入扩展

    # 如果直接成功，说明之前已超时回退；如果失败 0x7F，说明仍在扩展
    return {
        'test': 'S3_timeout_revert',
        'result': 'PASS' if result['success'] else 'VERIFY_MANUALLY'
```

```
        }
```

3.3.2 安全访问服务（$27 Seed&Key 验证）

完整安全访问测试框架:

```
class SecurityAccessTest:
    def __init__(self, diag, seed_key_dll):
        self.diag = diag
        self.seed_key = ctypes.CDLL(seed_key_dll)

    def test_security_levels(self):
        """ 测试多级安全访问"""
        results = []

        for level in [1, 3, 5, 7]:  # 典型安全等级
            results.append(self._test_single_level(level))

        return results

    def _test_single_level(self, level):
        """ 测试单个安全等级"""
        # 请求 Seed
        seed_response = self.diag.ExecuteService([0x27, level * 2 - 1])  # 0x01, 0x03, 0x05, 0x07

        if not seed_response['success']:
            return {
                'level': level,
                'phase': 'request_seed',
                'result': 'FAIL',
                'nrc': seed_response.get('nrc')
            }

        seed = seed_response['Data'][2:]  # 跳过 SID+ 子功能

        # 计算 Key
        key_buffer = (ctypes.c_ubyte * 16)()
        seed_array = (ctypes.c_ubyte * len(seed))(*seed)

        result = self.seed_key.GenerateKeyEx(
            seed_array, len(seed),
            key_buffer, 16,
            level, b"", b"ECU1"
        )

        if result != 0:
            return {
                'level': level,
                'phase': 'compute_key',
                'result': 'ERROR',
                'error_code': result
            }
```

```python
        key = list(key_buffer[:len(seed)])

        # 发送 Key
        key_request = [0x27, level * 2] + key  # 0x02, 0x04, 0x06, 0x08
        key_response = self.diag.ExecuteService(key_request)

        # 验证解锁成功: 尝试执行受保护服务
        verification = self._verify_unlocked(level)

        return {
            'level': level,
            'seed': seed.hex(),
            'key': bytes(key).hex(),
            'unlock_result': 'PASS' if key_response['success'] else 'FAIL',
            'verification': verification
        }

    def _verify_unlocked(self, level):
        """ 验证解锁状态: 尝试写入受保护 DID"""
        # 尝试写入测试 DID (需根据实际 ECU 配置)
        test_write = self.diag.ExecuteService(
            [0x2E, 0xF1, 0x90] + [0x00] * 17  # 尝试写 VIN
        )

        if test_write['success']:
            return {'method': 'write_protected_did', 'result': 'UNLOCKED'}

        nrc = test_write.get('nrc')
        if nrc == 0x33:  # SecurityAccessDenied
            return {'method': 'write_protected_did', 'result': 'LOCKED'}

        return {'method': 'write_protected_did', 'result': 'UNKNOWN', 'nrc': nrc}

    def test_brute_force_protection(self, level=1):
        """ 测试暴力破解防护机制"""
        results = []

        # 连续发送错误 Key
        for attempt in range(10):
            # 请求新 Seed
            seed_resp = self.diag.ExecuteService([0x27, level * 2 - 1])
            if not seed_resp['success']:
                break  # 可能已锁定

            seed = seed_resp['Data'][2:]

            # 发送错误 Key (全 0)
            wrong_key = [0x00] * len(seed)
            key_request = [0x27, level * 2] + wrong_key
            response = self.diag.ExecuteService(key_request)
```

```
            results.append({
                'attempt': attempt + 1,
                'nrc': response.get('nrc'),
                'locked': response.get('nrc') == 0x36  # ExceedNumberOfAttempts
            })

            if response.get('nrc') in [0x36, 0x37]:  # 锁定或延迟
                break

        return {
            'test': 'brute_force_protection',
            'level': level,
            'attempts_to_lock': len(results),
            'details': results
        }
```

3.3.3 数据读写服务（$22 ReadDataByIdentifier/$2E WriteDataByIdentifier） DID 测试数据驱动框架:

```
class DataIdentifierTest:
    def __init__(self, diag, did_database):
        self.diag = diag
        self.did_db = did_database  # 从 ODX/JSON 加载的 DID 定义

    def test_all_dids(self, test_type='read'):
        """ 批量测试所有 DID"""
        results = []

        for did_def in self.did_db:
            if test_type == 'read' and did_def['access'] in ['R', 'RW']:
                result = self._test_read_did(did_def)
            elif test_type == 'write' and did_def['access'] in ['W', 'RW']:
                result = self._test_write_did(did_def)
            else:
                continue

            results.append(result)

        return results

    def _test_read_did(self, did_def):
        """ 测试 DID 读取"""
        did_bytes = did_def['id'].to_bytes(2, 'big')
        request = [0x22] + list(did_bytes)

        response = self.diag.ExecuteService(request)

        if not response['success']:
            return {
                'did': did_def['id'],
                'name': did_def['name'],
```

```
            'result': 'FAIL',
            'phase': 'read',
            'nrc': response.get('nrc')
        }

    data = response['Data'][3:]  # 跳过 SID+DID

    # 数据解析和验证
    parsed = self._parse_did_data(data, did_def['format'])
    validation = self._validate_parsed_data(parsed, did_def['constraints'])

    return {
        'did': did_def['id'],
        'name': did_def['name'],
        'result': 'PASS' if validation['valid'] else 'FAIL',
        'raw_data': data.hex(),
        'parsed': parsed,
        'validation': validation
    }

def _test_write_did(self, did_def):
    """ 测试 DID 写入（含回读验证）"""
    # 生成测试数据
    test_data = self._generate_test_data(did_def['format'], 'nominal')

    did_bytes = did_def['id'].to_bytes(2, 'big')
    request = [0x2E] + list(did_bytes) + test_data

    write_response = self.diag.ExecuteService(request)

    if not write_response['success']:
        return {
            'did': did_def['id'],
            'result': 'FAIL',
            'phase': 'write',
            'nrc': write_response.get('nrc')
        }

    # 回读验证
    read_back = self._test_read_did(did_def)

    match = (read_back.get('raw_data') == bytes(test_data).hex())

    return {
        'did': did_def['id'],
        'result': 'PASS' if match else 'FAIL',
        'phase': 'verify',
        'written': test_data.hex(),
        'read_back': read_back.get('raw_data'),
        'match': match
    }
```

```python
    def _parse_did_data(self, data, format_def):
        """ 根据格式定义解析 DID 数据"""
        result = {}
        offset = 0

        for field in format_def:
            if field['type'] == 'uint8':
                value = data[offset]
                offset += 1
            elif field['type'] == 'uint16':
                value = int.from_bytes(data[offset:offset+2], 'big')
                offset += 2
            elif field['type'] == 'ascii':
                length = field['length']
                value = bytes(data[offset:offset+length]).decode('ascii')
                offset += length
            # ... 更多类型

            result[field['name']] = {
                'value': value,
                'unit': field.get('unit', '')
            }

        return result
```

3.3.4 故障码读取与清除（$19 ReadDTC/$14 ClearDTC）

DTC 生命周期完整测试:

```python
class DTCTest:
    def __init__(self, diag, dtc_database):
        self.diag = diag
        self.dtc_db = dtc_database

    def test_dtc_lifecycle(self):
        """ 测试 DTC 完整生命周期"""
        results = []

        # 阶段 1: 清除所有 DTC，确保干净状态
        clear_result = self._clear_all_dtcs()
        results.append({'phase': 'clear_initial', **clear_result})

        # 阶段 2: 注入故障，验证检测
        for dtc_def in self.dtc_db[:5]:  # 抽样测试
            injection = self._inject_fault(dtc_def)
            detection = self._verify_dtc_detected(dtc_def)

            results.append({
                'phase': 'detection',
                'dtc': dtc_def['code'],
                'injection_method': injection['method'],
                'detected': detection['found'],
```

```
            'status_byte': detection.get('status')
        })

    # 阶段 3: 读取 DTC 详细信息
    dtc_info = self._read_dtc_information()
    results.append({'phase': 'read_detailed', **dtc_info})

    # 阶段 4: 修复故障，验证状态迁移
    recovery = self._simulate_fault_recovery()
    status_migration = self._verify_status_migration()

    results.append({
        'phase': 'status_migration',
        'test_failed_cleared': status_migration.get('tf_cleared'),
        'confirmed_pending': status_migration.get('confirmed_to_pending')
    })

    # 阶段 5: 清除 DTC，验证清除
    final_clear = self._clear_all_dtcs()
    verification = self._verify_no_dtcs()

    results.append({
        'phase': 'final_clear',
        'clear_success': final_clear['success'],
        'verification_pass': verification['no_dtcs']
    })

    return results

def _clear_all_dtcs(self):
    """ 执行 $14 清除所有 DTC"""
    # 功能请求方式（广播）或物理请求方式
    response = self.diag.ExecuteService([0x14, 0xFF, 0xFF, 0xFF])
    return {'success': response['success'], 'nrc': response.get('nrc')}

def _inject_fault(self, dtc_def):
    """ 注入指定故障条件"""
    methods = {
        'sensor_open_circuit': lambda: self._set_sensor_value(dtc_def['sensor'], 0xFFFF),
        'sensor_short_to_ground': lambda: self._set_sensor_value(dtc_def['sensor'], 0x0000),
        'communication_timeout': lambda: self._disable_node_communication(dtc_def['node']),
        'plausibility_error': lambda: self._set_implausible_sensor_combination(dtc_def)
    }

    method = dtc_def.get('injection_method', 'sensor_open_circuit')
    injector = methods.get(method, methods['sensor_open_circuit'])
    injector()

    # 等待故障检测去抖动
    time.sleep(dtc_def.get('debounce_time_ms', 100) / 1000.0)
```

```python
        return {'method': method, 'executed': True}

    def _verify_dtc_detected(self, dtc_def):
        """ 验证指定 DTC 已被检测记录"""
        # 使用 $19 02 读取 DTC 列表
        response = self.diag.ExecuteService([0x19, 0x02, 0xFF])

        if not response['success']:
            return {'found': False, 'error': response.get('nrc')}

        # 解析 DTC 列表
        dtcs = self._parse_dtc_list(response['Data'])

        target_dtc = dtc_def['code']
        for dtc in dtcs:
            if dtc['code'] == target_dtc:
                return {
                    'found': True,
                    'status': dtc['status_byte'],
                    'test_failed': bool(dtc['status_byte'] & 0x01),
                    'confirmed': bool(dtc['status_byte'] & 0x08)
                }

        return {'found': False, 'searched': target_dtc}

    def _read_dtc_information(self):
        """ 读取完整 DTC 信息（数量、列表、快照、扩展数据）"""
        info = {}

        # $19 01: DTC 数量
        r01 = self.diag.ExecuteService([0x19, 0x01, 0xFF])
        info['count_by_status'] = self._parse_dtc_count(r01.get('Data', []))

        # $19 02: DTC 列表
        r02 = self.diag.ExecuteService([0x19, 0x02, 0xFF])
        info['dtc_list'] = self._parse_dtc_list(r02.get('Data', []))

        # $19 04: 快照数据（对每个 confirmed DTC）
        for dtc in info['dtc_list']:
            if dtc.get('confirmed'):
                r04 = self.diag.ExecuteService([
                    0x19, 0x04,
                    *dtc['code_bytes'],  # DTC 编号
                    0xFF  # 所有快照记录
                ])
                dtc['snapshots'] = self._parse_snapshots(r04.get('Data', []))

        # $19 06: 扩展数据
        for dtc in info['dtc_list']:
            r06 = self.diag.ExecuteService([
                0x19, 0x06,
```

```
                *dtc['code_bytes'],
                0xFF  # 所有扩展数据记录
            ])
            dtc['extended_data'] = self._parse_extended_data(r06.get('Data', []))

        return info
```

3.3.5 ECU 刷写流程（$34 RequestDownload/$36 TransferData/$37 RequestTransferExit） 完整 BootLoader 自动化测试:

```
class ECUFlashingTest:
    def __init__(self, diag, firmware_file, flash_config):
        self.diag = diag
        self.firmware = self._load_firmware(firmware_file)
        self.config = flash_config  # 地址、擦除策略、校验算法等

    def execute_full_flash_process(self):
        """ 执行完整刷写流程"""
        phases = [
            ('preparation', self._preparation_phase),
            ('erasure', self._erasure_phase),
            ('download', self._download_phase),
            ('verification', self._verification_phase),
            ('completion', self._completion_phase)
        ]

        results = []
        for name, phase_func in phases:
            result = phase_func()
            results.append({'phase': name, **result})

            if not result.get('success'):
                # 执行错误恢复
                recovery = self._error_recovery(name, result)
                results.append({'phase': f'{name}_recovery', **recovery})

                if not recovery.get('success'):
                    break  # 无法恢复，终止流程

        return results

    def _preparation_phase(self):
        """ 预编程阶段"""
        steps = []

        # 1. 进入扩展会话
        r1 = self.diag.ExecuteService([0x10, 0x03])
        steps.append({'step': 'enter_extended', 'result': r1['success'], 'nrc': r1.get('nrc')})

        # 2. 控制 DTC 设置（关闭）
        r2 = self.diag.ExecuteService([0x85, 0x02])
```

```
        steps.append({'step': 'disable_dtc', 'result': r2['success']})

        # 3. 控制通信（禁用非诊断通信）
        r3 = self.diag.ExecuteService([0x28, 0x03, 0x03])  # 禁用 Tx 和 Rx
        steps.append({'step': 'disable_communication', 'result': r3['success']})

        # 4. 安全访问解锁
        r4 = self._unlock_security_level(self.config['security_level'])
        steps.append({'step': 'security_unlock', 'result': r4['success']})

        # 5. 写入指纹信息（如需要）
        if self.config.get('fingerprint_did'):
            fingerprint = self._generate_fingerprint()
            r5 = self.diag.ExecuteService(
                [0x2E] + list(self.config['fingerprint_did'].to_bytes(2, 'big')) + fingerprint
            )
            steps.append({'step': 'write_fingerprint', 'result': r5['success']})

        # 6. 进入编程会话
        r6 = self.diag.ExecuteService([0x10, 0x02])
        steps.append({'step': 'enter_programming', 'result': r6['success']})

        # 7. 再次安全访问（编程会话可能需重新解锁）
        r7 = self._unlock_security_level(self.config['programming_security_level'])
        steps.append({'step': 'unlock_programming', 'result': r7['success']})

        success = all(s['result'] for s in steps)
        return {'success': success, 'steps': steps}

    def _download_phase(self):
        """ 数据传输阶段"""
        # 解析固件为数据块
        blocks = self._segment_firmware(
            self.firmware,
            self.config['base_address'],
            self.config['block_size']
        )

        download_results = []

        for block_idx, (address, data) in enumerate(blocks):
            block_result = self._download_single_block(block_idx, address, data)
            download_results.append(block_result)

            # 进度报告
            progress = (block_idx + 1) / len(blocks) * 100
            self._report_progress('download', progress)

            if not block_result['success']:
                return {
                    'success': False,
```

```
                'failed_block': block_idx,
                'error': block_result.get('error'),
                'completed_blocks': block_idx
            }

    return {
        'success': True,
        'total_blocks': len(blocks),
        'total_bytes': len(self.firmware),
        'block_details': download_results
    }

def _download_single_block(self, block_idx, address, data):
    """ 下载单个数据块: $34-$36-$37 序列"""
    # $34 RequestDownload
    format_id = 0x00  # 压缩和加密标识
    addr_len_format = 0x44  # 4 字节地址, 4 字节长度

    request_34 = [0x34, format_id, addr_len_format]
    request_34 += list(address.to_bytes(4, 'big'))
    request_34 += list(len(data).to_bytes(4, 'big'))

    r34 = self.diag.ExecuteService(request_34, timeout=5000)  # 擦除可能耗时
    if not r34['success']:
        return {'success': False, 'phase': 'request_download', 'nrc': r34.get('nrc')}

    # 提取最大块大小 (如 ECU 有约束)
    max_block_size = self._parse_max_block_size(r34.get('Data', []))

    # $36 TransferData (可能分多个连续帧)
    sequence_counter = 1
    offset = 0

    while offset < len(data):
        chunk = data[offset:offset + max_block_size]

        request_36 = [0x36, sequence_counter] + list(chunk)
        r36 = self.diag.ExecuteService(request_36, timeout=1000)

        if not r36['success']:
            return {
                'success': False,
                'phase': 'transfer_data',
                'sequence': sequence_counter,
                'nrc': r36.get('nrc')
            }

        sequence_counter = (sequence_counter + 1) & 0xFF  # 模 256 环绕
        offset += len(chunk)

    # $37 RequestTransferExit
```

```python
        request_37 = [0x37]
        # 如需要，附加校验和
        if self.config.get('checksum_in_37'):
            checksum = self._calculate_checksum(data, self.config['checksum_algorithm'])
            request_37 += list(checksum)

        r37 = self.diag.ExecuteService(request_37, timeout=2000)

        return {
            'success': r37['success'],
            'phase': 'transfer_exit',
            'nrc': r37.get('nrc'),
            'bytes_transferred': len(data)
        }

    def _verification_phase(self):
        """ 软件完整性验证阶段"""
        methods = []

        # 方法 1: $31 例程控制执行完整性检查
        if self.config.get('verify_routine_id'):
            r31 = self.diag.ExecuteService([
                0x31, 0x01,  # 启动例程
                *self.config['verify_routine_id'].to_bytes(2, 'big')
            ])
            methods.append({
                'method': 'routine_control',
                'result': r31['success'],
                'routine_result': self._parse_routine_result(r31.get('Data', []))
            })

        # 方法 2: $22 读取软件版本/校验和 DID
        if self.config.get('version_did'):
            r22 = self.diag.ExecuteService(
                [0x22] + list(self.config['version_did'].to_bytes(2, 'big'))
            )
            if r22['success']:
                read_version = r22['Data'][3:]
                expected_version = self.config.get('expected_version')
                methods.append({
                    'method': 'version_check',
                    'result': read_version == expected_version,
                    'read': read_version.hex(),
                    'expected': expected_version.hex() if expected_version else None
                })

        # 方法 3: 启动应用后功能验证（如需要）
        if self.config.get('post_flash_test'):
            # 复位 ECU，验证启动
            reset_result = self._reset_and_verify()
            methods.append({
```

```
                'method': 'post_flash_functional',
                'result': reset_result['success']
            })

        return {
            'success': all(m['result'] for m in methods),
            'methods': methods
        }
```

3.4 UDS 自动化测试脚本实例

3.4.1 基于系统变量的诊断触发与响应读取 生产级系统变量驱动框架:

```
class SystemVariableDiagnosticDriver:
    """
    基于 TSMaster 系统变量的诊断驱动层
    优势: 异步解耦, 测试逻辑与通信时序分离
    """

    # 系统变量命名规范
    VAR_PREFIX = "UDS_ECU1"
    VAR_REQUEST_TRIGGER = f"{VAR_PREFIX}_ReqTrigger"
    VAR_REQUEST_SID = f"{VAR_PREFIX}_ReqSID"
    VAR_REQUEST_DATA = f"{VAR_PREFIX}_ReqData"
    VAR_RESPONSE_READY = f"{VAR_PREFIX}_RspReady"
    VAR_RESPONSE_TYPE = f"{VAR_PREFIX}_RspType"  # 0= 等待, 1= 肯定, 2= 否定, 3= 挂起
    VAR_RESPONSE_DATA = f"{VAR_PREFIX}_RspData"
    VAR_RESPONSE_NRC = f"{VAR_PREFIX}_RspNRC"
    VAR_PROGRESS_PERCENT = f"{VAR_PREFIX}_Progress"

    def __init__(self, app, ecu_name="ECU1"):
        self.app = app
        self.sys_vars = app.SystemVariables
        self.ecu_name = ecu_name
        self._ensure_variables()
        self._setup_event_handler()

    def _ensure_variables(self):
        """ 确保所有系统变量存在"""
        var_defs = [
            (self.VAR_REQUEST_TRIGGER, 'UInt8', 0),
            (self.VAR_REQUEST_SID, 'UInt8', 0),
            (self.VAR_REQUEST_DATA, 'UInt8Array', []),
            (self.VAR_RESPONSE_READY, 'UInt8', 0),
            (self.VAR_RESPONSE_TYPE, 'UInt8', 0),
            (self.VAR_RESPONSE_DATA, 'UInt8Array', []),
            (self.VAR_RESPONSE_NRC, 'UInt8', 0),
            (self.VAR_PROGRESS_PERCENT, 'UInt8', 0),
        ]

        for name, vtype, init in var_defs:
```

```python
        try:
            self.sys_vars.Get(name)
        except:
            self.sys_vars.Add(name, vtype, init)

    def _setup_event_handler(self):
        """ 设置响应就绪事件处理"""
        # 使用 TSMaster 事件机制或轮询
        self.response_event = threading.Event()

    def execute_service(self, request_data: list, timeout_ms: int = 5000) -> dict:
        """
        执行诊断服务，通过系统变量交互

        Args:
            request_data: 完整请求数据，首字节为 SID
            timeout_ms: 最大等待时间

        Returns:
            包含 success, response_type, data, nrc 的字典
        """
        # 重置状态
        self.sys_vars.Set(self.VAR_RESPONSE_READY, 0)
        self.sys_vars.Set(self.VAR_RESPONSE_TYPE, 0)
        self.response_event.clear()

        # 写入请求参数
        self.sys_vars.Set(self.VAR_REQUEST_SID, request_data[0])
        if len(request_data) > 1:
            self.sys_vars.Set(self.VAR_REQUEST_DATA, request_data[1:])

        # 触发执行
        self.sys_vars.Set(self.VAR_REQUEST_TRIGGER, 1)

        # 等待响应（事件或轮询）
        if not self.response_event.wait(timeout_ms / 1000.0):
            return {
                'success': False,
                'error': 'TIMEOUT',
                'timeout_ms': timeout_ms
            }

        # 读取响应
        response_type = self.sys_vars.Get(self.VAR_RESPONSE_TYPE)
        result = {
            'success': response_type == 1,  # 1= 肯定响应
            'response_type': ['WAITING', 'POSITIVE', 'NEGATIVE', 'PENDING'][response_type],
            'data': list(self.sys_vars.Get(self.VAR_RESPONSE_DATA)),
            'nrc': self.sys_vars.Get(self.VAR_RESPONSE_NRC)
        }
```

```python
        return result

    def execute_flow(self, flow_name: str, params: dict = None,
                     progress_callback=None) -> dict:
        """
        执行预配置的 UDS Flow

        Args:
            flow_name: TSMaster 中配置的流程名称
            params: 流程参数（如替换变量）
            progress_callback: 进度更新回调函数

        Returns:
            流程执行结果
        """
        # 设置流程参数
        if params:
            for key, value in params.items():
                self.sys_vars.Set(f"{self.VAR_PREFIX}_Param_{key}", value)

        # 触发流程
        self.sys_vars.Set(f"{self.VAR_PREFIX}_FlowTrigger", flow_name)

        # 监控进度
        last_progress = 0
        while True:
            progress = self.sys_vars.Get(self.VAR_PROGRESS_PERCENT)
            status = self.sys_vars.Get(f"{self.VAR_PREFIX}_FlowStatus")

            if progress_callback and progress != last_progress:
                progress_callback(progress, status)
                last_progress = progress

            if status in ['COMPLETED', 'FAILED', 'ABORTED']:
                break

            time.sleep(0.1)

        return {
            'success': status == 'COMPLETED',
            'final_status': status,
            'result_vars': self._collect_result_variables()
        }

    def _collect_result_variables(self) -> dict:
        """ 收集流程结果变量"""
        prefix = f"{self.VAR_PREFIX}_Result_"
        results = {}

        # 遍历所有匹配的结果变量
        for var_name in self._list_variables_with_prefix(prefix):
```

```
            results[var_name[len(prefix):]] = self.sys_vars.Get(var_name)

        return results
```

3.4.2 诊断流程状态机实现 完整刷写状态机（生产级）：

```
from enum import Enum, auto
from dataclasses import dataclass
from typing import Callable, Optional

class FlashState(Enum):
    """ 刷写流程状态枚举"""
    IDLE = auto()
    INITIALIZING = auto()
    PREPARING = auto()
    ERASING = auto()
    DOWNLOADING = auto()
    VERIFYING = auto()
    FINALIZING = auto()
    COMPLETED = auto()
    ERROR = auto()
    RECOVERING = auto()

@dataclass
class StateTransition:
    """ 状态转换定义"""
    from_state: FlashState
    event: str
    to_state: FlashState
    action: Optional[Callable] = None
    guard: Optional[Callable] = None

class FlashingStateMachine:
    """
    ECU 刷写流程状态机
    支持正交区域：主流程 + 并行监控
    """

    def __init__(self, driver: SystemVariableDiagnosticDriver,
                 firmware: bytes, config: dict):
        self.driver = driver
        self.firmware = firmware
        self.config = config

        self.state = FlashState.IDLE
        self.context = {
            'retry_count': 0,
            'max_retries': 3,
            'current_block': 0,
            'total_blocks': 0,
            'transferred_bytes': 0,
```

```
            'error_code': None,
            'recovery_attempts': 0
        }

        self._build_transition_table()
        self._enter_state(FlashState.IDLE)

    def _build_transition_table(self):
        """ 构建状态转换表"""
        self.transitions = [
            # 启动
            StateTransition(FlashState.IDLE, 'start', FlashState.INITIALIZING,
                            self._on_initialize),

            # 初始化成功
            StateTransition(FlashState.INITIALIZING, 'success', FlashState.PREPARING,
                            self._on_prepare),
            StateTransition(FlashState.INITIALIZING, 'error', FlashState.ERROR,
                            self._on_init_error),

            # 准备阶段
            StateTransition(FlashState.PREPARING, 'success', FlashState.ERASING,
                            self._on_erase),
            StateTransition(FlashState.PREPARING, 'error', FlashState.RECOVERING,
                            self._on_recover,
                            guard=lambda ctx: ctx['retry_count'] < ctx['max_retries']),
            StateTransition(FlashState.PREPARING, 'error', FlashState.ERROR,
                            guard=lambda ctx: ctx['retry_count'] >= ctx['max_retries']),

            # 擦除阶段
            StateTransition(FlashState.ERASING, 'success', FlashState.DOWNLOADING,
                            self._on_download_start),
            StateTransition(FlashState.ERASING, 'error', FlashState.RECOVERING,
                            self._on_recover),

            # 下载阶段（内部循环）
            StateTransition(FlashState.DOWNLOADING, 'block_complete', FlashState.DOWNLOADING,
                            self._on_block_complete),
            StateTransition(FlashState.DOWNLOADING, 'all_blocks_complete', FlashState.VERIFYING,
                            self._on_verify),
            StateTransition(FlashState.DOWNLOADING, 'block_error', FlashState.RECOVERING,
                            self._on_block_recover),

            # 验证阶段
            StateTransition(FlashState.VERIFYING, 'success', FlashState.FINALIZING,
                            self._on_finalize),
            StateTransition(FlashState.VERIFYING, 'error', FlashState.ERROR,
                            self._on_verify_fail),

            # 完成阶段
            StateTransition(FlashState.FINALIZING, 'success', FlashState.COMPLETED,
```

```
                        self._on_complete),

        # 错误恢复
        StateTransition(FlashState.RECOVERING, 'success', FlashState.PREPARING,
                        self._on_recovery_success),
        StateTransition(FlashState.RECOVERING, 'failed', FlashState.ERROR,
                        self._on_recovery_failed),
    ]

    # 构建快速查找索引
    self._transition_index = {}
    for t in self.transitions:
        key = (t.from_state, t.event)
        if key not in self._transition_index:
            self._transition_index[key] = []
        self._transition_index[key].append(t)

def trigger(self, event: str, event_data: dict = None):
    """ 触发状态转换"""
    # 查找匹配转换
    key = (self.state, event)
    candidates = self._transition_index.get(key, [])

    # 评估 guard 条件
    valid_transitions = [
        t for t in candidates
        if t.guard is None or t.guard(self.context)
    ]

    if not valid_transitions:
        self._log(f"No valid transition for {self.state.name} + {event}")
        return False

    # 选择优先级最高的转换（简单实现：第一个）
    transition = valid_transitions[0]

    # 执行退出动作
    self._exit_state(self.state)

    # 状态转移
    old_state = self.state
    self.state = transition.to_state

    # 执行进入动作
    self._enter_state(self.state)

    # 执行转换动作
    if transition.action:
        try:
            transition.action(event_data)
        except Exception as e:
```

```
            self._log(f"Action failed: {e}")
            self.trigger('error', {'original_error': e})
            return False

    self._log(f"Transition: {old_state.name} --{event}--> {self.state.name}")
    return True

def _enter_state(self, state: FlashState):
    """ 进入状态的标准动作"""
    entry_actions = {
        FlashState.INITIALIZING: self._entry_initializing,
        FlashState.PREPARING: self._entry_preparing,
        FlashState.ERASING: self._entry_erasing,
        FlashState.DOWNLOADING: self._entry_downloading,
        FlashState.VERIFYING: self._entry_verifying,
        FlashState.FINALIZING: self._entry_finalizing,
        FlashState.ERROR: self._entry_error,
        FlashState.RECOVERING: self._entry_recovering,
    }

    action = entry_actions.get(state)
    if action:
        action()

def _exit_state(self, state: FlashState):
    """ 退出状态的标准动作"""
    # 清理定时器、取消订阅等
    pass

# ========== 具体状态动作实现 ==========

def _entry_initializing(self):
    """ 初始化进入：加载固件，计算参数"""
    self.context['total_blocks'] = (
        len(self.firmware) + self.config['block_size'] - 1
    ) // self.config['block_size']
    self._report_progress(0, "Initializing...")

def _on_initialize(self, data):
    """ 初始化动作：验证固件格式"""
    # 验证 S19/HEX/BIN 格式
    if not self._validate_firmware_format():
        raise ValueError("Invalid firmware format")
    self.trigger('success')

def _entry_preparing(self):
    """ 准备阶段进入：执行预编程步骤"""
    self._report_progress(5, "Preparing ECU...")

    # 异步执行准备序列
    threading.Thread(target=self._execute_preparation_sequence).start()
```

```python
def _execute_preparation_sequence(self):
    """ 执行准备序列（在后台线程）"""
    steps = [
        (0x10, [0x03], "Enter extended session"),
        (0x85, [0x02], "Disable DTC"),
        (0x28, [0x03, 0x03], "Disable communication"),
        (0x27, [0x01], "Request seed"),  # 需特殊处理
    ]

    for sid, data, desc in steps:
        result = self.driver.execute_service([sid] + data, timeout=5000)

        if not result['success']:
            self.context['error_code'] = result.get('nrc', 'UNKNOWN')
            self.trigger('error', {'failed_step': desc, 'result': result})
            return

        # 安全访问特殊处理
        if sid == 0x27:
            key = self._compute_key(result['data'])
            key_result = self.driver.execute_service([0x27, 0x02] + key)
            if not key_result['success']:
                self.trigger('error', {'failed_step': 'Send key'})
                return

    self.trigger('success')

def _entry_downloading(self):
    """ 下载阶段进入：启动块传输"""
    self._start_block_transfer()

def _start_block_transfer(self):
    """ 启动单块传输（递归/循环实现）"""
    block_idx = self.context['current_block']

    if block_idx >= self.context['total_blocks']:
        self.trigger('all_blocks_complete')
        return

    # 计算块参数
    offset = block_idx * self.config['block_size']
    block_data = self.firmware[offset:offset + self.config['block_size']]
    address = self.config['base_address'] + offset

    # 异步执行块下载
    threading.Thread(
        target=self._execute_block_download,
        args=(block_idx, address, block_data)
    ).start()
```

```
        progress = 10 + int(80 * block_idx / self.context['total_blocks'])
        self._report_progress(progress, f"Downloading block
        ↪  {block_idx+1}/{self.context['total_blocks']}")

    def _execute_block_download(self, block_idx, address, data):
        """ 执行单块下载序列"""
        try:
            # $34 RequestDownload
            r34 = self._request_download(address, len(data))
            if not r34['success']:
                self.trigger('block_error', {'phase': '34', 'nrc': r34.get('nrc')})
                return

            max_size = self._parse_max_block_size(r34.get('data', []))

            # $36 TransferData (可能分片)
            sequence = 1
            offset = 0
            while offset < len(data):
                chunk = data[offset:offset + max_size]
                r36 = self._transfer_data(sequence, chunk)

                if not r36['success']:
                    self.trigger('block_error', {
                        'phase': '36',
                        'sequence': sequence,
                        'nrc': r36.get('nrc')
                    })
                    return

                sequence = (sequence + 1) & 0xFF
                offset += len(chunk)

                # 更新进度
                sub_progress = int(80 * offset / len(self.firmware))
                self._report_progress(10 + sub_progress, f"Block {block_idx+1}: {offset}/{len(data)}")

            # $37 RequestTransferExit
            r37 = self._request_transfer_exit(data)
            if not r37['success']:
                self.trigger('block_error', {'phase': '37', 'nrc': r37.get('nrc')})
                return

            # 块完成
            self.context['current_block'] += 1
            self.context['transferred_bytes'] += len(data)
            self.trigger('block_complete')

        except Exception as e:
            self.trigger('block_error', {'exception': str(e)})
```

```python
    def _report_progress(self, percent: int, message: str):
        """ 报告进度"""
        # 更新系统变量
        self.driver.sys_vars.Set(self.driver.VAR_PROGRESS_PERCENT, percent)
        # 回调
        if self.config.get('progress_callback'):
            self.config['progress_callback'](percent, message)

    def run_to_completion(self) -> dict:
        """ 运行状态机至完成"""
        self.trigger('start')

        # 等待完成（实际实现需更完善的同步机制）
        max_wait = 600  # 10 分钟超时
        for _ in range(max_wait * 10):
            if self.state in [FlashState.COMPLETED, FlashState.ERROR]:
                break
            time.sleep(0.1)

        return {
            'success': self.state == FlashState.COMPLETED,
            'final_state': self.state.name,
            'context': self.context
        }
```

3.4.3 批量诊断服务序列执行 数据驱动的批量测试框架:

```python
import csv
import json
from concurrent.futures import ThreadPoolExecutor, as_completed

class BatchDiagnosticExecutor:
    """
    批量诊断测试执行引擎
    支持：顺序执行、并行执行、参数化测试、结果聚合
    """

    def __init__(self, driver: SystemVariableDiagnosticDriver,
                 max_workers: int = 4):
        self.driver = driver
        self.max_workers = max_workers
        self.results = []

    def load_test_cases(self, source: str, format: str = 'json') -> list:
        """ 加载测试用例"""
        if format == 'json':
            with open(source, 'r', encoding='utf-8') as f:
                data = json.load(f)
                return data.get('test_cases', [])

        elif format == 'csv':
```

```python
        cases = []
        with open(source, 'r', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row in reader:
                cases.append(self._parse_csv_row(row))
        return cases

    elif format == 'odx':
        # 从 ODX 数据库提取测试用例
        return self._extract_from_odx(source)

    else:
        raise ValueError(f"Unsupported format: {format}")

def execute_sequential(self, cases: list,
                       stop_on_failure: bool = False,
                       progress_callback=None) -> list:
    """ 顺序执行测试用例"""
    results = []

    for idx, case in enumerate(cases):
        if progress_callback:
            progress_callback(idx + 1, len(cases), case.get('id', f'Case_{idx}'))

        result = self._execute_single(case)
        results.append(result)

        if stop_on_failure and result['status'] == 'FAIL':
            print(f"Stopping on failure: {case.get('id')}")
            break

    self.results.extend(results)
    return results

def execute_parallel(self, cases: list,
                     grouping_strategy: str = 'by_ecu') -> list:
    """
    并行执行测试用例

    grouping_strategy:
    - 'by_ecu': 同一 ECU 的用例顺序执行，不同 ECU 并行
    - 'independent': 完全并行（需 ECU 支持并发）
    - 'none': 退化为顺序执行
    """
    if grouping_strategy == 'none':
        return self.execute_sequential(cases)

    # 按 ECU 分组
    groups = {}
    for case in cases:
        ecu = case.get('ecu', 'default')
```

```
            groups.setdefault(ecu, []).append(case)

        # 每组内顺序，组间并行
        with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
            futures = {
                executor.submit(self.execute_sequential, group_cases): ecu
                for ecu, group_cases in groups.items()
            }

            all_results = []
            for future in as_completed(futures):
                ecu = futures[future]
                try:
                    results = future.result()
                    all_results.extend(results)
                except Exception as e:
                    print(f"ECU {ecu} execution failed: {e}")

        self.results.extend(all_results)
        return all_results

    def _execute_single(self, case: dict) -> dict:
        """ 执行单个测试用例"""
        start_time = time.time()
        case_id = case.get('id', 'unknown')

        try:
            # 前置条件
            if not self._check_precondition(case.get('precondition')):
                return {
                    'id': case_id,
                    'status': 'SKIPPED',
                    'reason': 'precondition_not_met',
                    'duration_ms': 0
                }

            # 构建请求
            request = self._build_request(case)

            # 执行服务
            timeout = case.get('timeout_ms', 5000)
            response = self.driver.execute_service(request, timeout)

            # 验证响应
            validation = self._validate_response(response, case.get('expected', {}))

            # 后置动作
            self._execute_post_action(case.get('post_action'))

            duration = (time.time() - start_time) * 1000
```

```python
        return {
            'id': case_id,
            'status': 'PASS' if validation['valid'] else 'FAIL',
            'request': request.hex() if isinstance(request, bytes) else request,
            'response': response,
            'validation': validation,
            'duration_ms': duration
        }

    except Exception as e:
        return {
            'id': case_id,
            'status': 'ERROR',
            'error': str(e),
            'duration_ms': (time.time() - start_time) * 1000
        }

def _build_request(self, case: dict) -> list:
    """ 从用例定义构建诊断请求"""
    sid = case['service']

    # 参数替换（支持变量）
    params = case.get('parameters', [])
    resolved_params = []
    for p in params:
        if isinstance(p, str) and p.startswith('$'):
            # 变量引用，从上下文或系统变量获取
            resolved_params.append(self._resolve_variable(p))
        else:
            resolved_params.append(p)

    # DID 处理（2 字节大端）
    if 'did' in case:
        did_bytes = case['did'].to_bytes(2, 'big')
        resolved_params = list(did_bytes) + resolved_params

    return [sid] + resolved_params

def _validate_response(self, response: dict, expected: dict) -> dict:
    """ 验证响应是否符合预期"""
    checks = []

    # 响应类型检查
    if 'response_type' in expected:
        actual_type = 'positive' if response.get('success') else 'negative'
        checks.append({
            'check': 'response_type',
            'expected': expected['response_type'],
            'actual': actual_type,
            'pass': actual_type == expected['response_type']
        })
```

```python
        # NRC 检查（否定响应时）
        if 'nrc' in expected:
            actual_nrc = response.get('nrc', 'N/A')
            checks.append({
                'check': 'nrc',
                'expected': expected['nrc'],
                'actual': actual_nrc,
                'pass': actual_nrc == expected['nrc']
            })

        # 数据内容检查
        if 'data_contains' in expected:
            actual_data = bytes(response.get('data', []))
            expected_pattern = bytes(expected['data_contains'])
            checks.append({
                'check': 'data_contains',
                'expected': expected_pattern.hex(),
                'actual': actual_data.hex(),
                'pass': expected_pattern in actual_data
            })

        # 数据长度检查
        if 'data_length' in expected:
            actual_len = len(response.get('data', []))
            checks.append({
                'check': 'data_length',
                'expected': expected['data_length'],
                'actual': actual_len,
                'pass': actual_len == expected['data_length']
            })

        # 信号值检查（需 DBC 支持）
        if 'signal_checks' in expected:
            for sig_check in expected['signal_checks']:
                # 从响应数据解析信号
                signal_value = self._extract_signal(
                    response.get('data', []),
                    sig_check['signal_path']
                )
                checks.append({
                    'check': f"signal:{sig_check['signal_path']}",
                    'expected_range': [sig_check.get('min'), sig_check.get('max')],
                    'actual': signal_value,
                    'pass': sig_check.get('min', float('-inf')) <= signal_value <= sig_check.get('max',
                    ↪  float('inf'))
                })

        all_pass = all(c['pass'] for c in checks)
        return {
            'valid': all_pass,
```

```
            'checks': checks,
            'failed_checks': [c for c in checks if not c['pass']]
        }

    def generate_report(self, output_path: str, format: str = 'html'):
        """ 生成测试报告"""
        summary = {
            'total': len(self.results),
            'passed': sum(1 for r in self.results if r['status'] == 'PASS'),
            'failed': sum(1 for r in self.results if r['status'] == 'FAIL'),
            'errors': sum(1 for r in self.results if r['status'] == 'ERROR'),
            'skipped': sum(1 for r in self.results if r['status'] == 'SKIPPED'),
            'total_duration_ms': sum(r.get('duration_ms', 0) for r in self.results)
        }

        if format == 'html':
            self._generate_html_report(output_path, summary, self.results)
        elif format == 'pdf':
            self._generate_pdf_report(output_path, summary, self.results)
        elif format == 'junit':
            self._generate_junit_xml(output_path, self.results)

        return summary
```

3.4.4 诊断结果日志记录与报告生成 企业级报告生成系统:

```
from datetime import datetime
import jinja2
import matplotlib.pyplot as plt
import io
import base64

class DiagnosticReportGenerator:
    """
    诊断测试报告生成器
    支持: HTML 交互式报告、PDF 打印报告、JUnit XML (CI 集成)、JSON 原始数据
    """

    HTML_TEMPLATE = """
    <!DOCTYPE html>
    <html>
    <head>
        <title>UDS 诊断测试报告 - {{ summary.timestamp }}</title>
        <style>
            body { font-family: Arial, sans-serif; margin: 40px; }
            .header { background: #1a5276; color: white; padding: 20px; }
            .summary { display: flex; gap: 20px; margin: 20px 0; }
            .metric { background: #f8f9fa; padding: 15px; border-radius: 8px; }
            .PASS { color: #28a745; font-weight: bold; }
            .FAIL { color: #dc3545; font-weight: bold; }
            .ERROR { color: #fd7e14; font-weight: bold; }
```

```
        table { width: 100%; border-collapse: collapse; margin: 20px 0; }
        th, td { padding: 12px; text-align: left; border-bottom: 1px solid #ddd; }
        th { background: #e9ecef; }
        tr:hover { background: #f5f5f5; }
        .details { display: none; }
        .toggle:checked + .details { display: block; }
        .chart { margin: 20px 0; text-align: center; }
    </style>
</head>
<body>
    <div class="header">
        <h1>UDS 诊断测试报告 </h1>
        <p> 生成时间: {{ summary.timestamp }} | 执行时长: {{ "%.2f"|format(summary.total_duration_ms/
         ↪  1000) }} 秒 </p>
    </div>

    <div class="summary">
        <div class="metric">
            <h3> 总计 </h3>
            <p style="font-size: 2em;">{{ summary.total }}</p>
        </div>
        <div class="metric">
            <h3 class="PASS"> 通过 </h3>
            <p style="font-size: 2em;" class="PASS">{{ summary.passed }}</p>
            <p>{{ "%.1f"|format(summary.passed/summary.total*100) }}%</p>
        </div>
        <div class="metric">
            <h3 class="FAIL"> 失败 </h3>
            <p style="font-size: 2em;" class="FAIL">{{ summary.failed }}</p>
        </div>
        <div class="metric">
            <h3 class="ERROR"> 错误 </h3>
            <p style="font-size: 2em;" class="ERROR">{{ summary.errors }}</p>
        </div>
    </div>

    <div class="chart">
        <img src="{{ charts.pie_chart }}"
             alt="结果分布" style="max-width: 400px;">
        <img src="{{ charts.timeline }}"
             alt="执行时序" style="max-width: 600px;">
    </div>

    <h2> 详细结果 </h2>
    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th> 服务 </th>
                <th> 状态 </th>
                <th> 耗时 (ms)</th>
```

```
                <th> 详情 </th>
            </tr>
        </thead>
        <tbody>
            {% for result in results %}
            <tr>
                <td>{{ result.id }}</td>
                <td>{{ result.service_name }}</td>
                <td class="{{ result.status }}">{{ result.status }}</td>
                <td>{{ "%.1f"|format(result.duration_ms) }}</td>
                <td>
                    <label>
                        <input type="checkbox" class="toggle">
                        查看详情
                        <div class="details">
                            <pre>{{ result.details_json }}</pre>
                        </div>
                    </label>
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>

    <h2> 失败分析 </h2>
    {% if summary.failed > 0 %}
    <table>
        <thead>
            <tr>
                <th>NRC</th>
                <th> 出现次数 </th>
                <th> 相关用例 </th>
            </tr>
        </thead>
        <tbody>
            {% for nrc, info in failure_analysis.nrc_distribution.items() %}
            <tr>
                <td>0x{{ "%02X"|format(nrc) }}</td>
                <td>{{ info.count }}</td>
                <td>{{ info.cases|join(', ') }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    {% else %}
    <p> 无失败记录 </p>
    {% endif %}
</body>
</html>
"""
```

```python
    def __init__(self):
        self.env = jinja2.Environment()

    def generate(self, results: list, output_path: str, format: str = 'html'):
        """ 生成指定格式的报告"""

        # 数据预处理
        summary = self._calculate_summary(results)
        charts = self._generate_charts(results) if format == 'html' else {}
        failure_analysis = self._analyze_failures(results)

        if format == 'html':
            template = self.env.from_string(self.HTML_TEMPLATE)
            html = template.render(
                summary=summary,
                results=self._enrich_results(results),
                charts=charts,
                failure_analysis=failure_analysis
            )
            with open(output_path, 'w', encoding='utf-8') as f:
                f.write(html)

        elif format == 'pdf':
            # 使用 weasyprint 或 pdfkit 转换 HTML
            html_path = output_path.replace('.pdf', '.html')
            self.generate(results, html_path, 'html')
            self._html_to_pdf(html_path, output_path)

        elif format == 'junit':
            self._generate_junit_xml(results, output_path)

        elif format == 'json':
            import json
            with open(output_path, 'w', encoding='utf-8') as f:
                json.dump({
                    'summary': summary,
                    'results': results,
                    'failure_analysis': failure_analysis
                }, f, indent=2, default=str)

        return output_path

    def _calculate_summary(self, results: list) -> dict:
        """ 计算汇总统计"""
        return {
            'timestamp': datetime.now().isoformat(),
            'total': len(results),
            'passed': sum(1 for r in results if r.get('status') == 'PASS'),
            'failed': sum(1 for r in results if r.get('status') == 'FAIL'),
            'errors': sum(1 for r in results if r.get('status') == 'ERROR'),
            'skipped': sum(1 for r in results if r.get('status') == 'SKIPPED'),
```

```
        'total_duration_ms': sum(r.get('duration_ms', 0) for r in results),
        'avg_duration_ms': sum(r.get('duration_ms', 0) for r in results) / len(results) if results
        ↪  else 0
    }

def _generate_charts(self, results: list) -> dict:
    """ 生成可视化图表（Base64 编码）"""
    charts = {}

    # 饼图：结果分布
    fig, ax = plt.subplots(figsize=(6, 6))
    status_counts = {}
    for r in results:
        status_counts[r.get('status', 'UNKNOWN')] = status_counts.get(r.get('status'), 0) + 1

    colors = {'PASS': '#28a745', 'FAIL': '#dc3545', 'ERROR': '#fd7e14', 'SKIPPED': '#6c757d'}
    ax.pie(status_counts.values(), labels=status_counts.keys(),
           colors=[colors.get(s, '#999') for s in status_counts.keys()],
           autopct='%1.1f%%', startangle=90)
    ax.set_title('Test Result Distribution')

    buf = io.BytesIO()
    plt.savefig(buf, format='png', bbox_inches='tight')
    charts['pie_chart'] = base64.b64encode(buf.getvalue()).decode()
    plt.close()

    # 时序图：执行时间线
    fig, ax = plt.subplots(figsize=(10, 4))
    times = [r.get('duration_ms', 0) for r in results]
    colors_list = [colors.get(r.get('status'), '#999') for r in results]

    ax.scatter(range(len(times)), times, c=colors_list, alpha=0.6)
    ax.set_xlabel('Test Case Index')
    ax.set_ylabel('Duration (ms)')
    ax.set_title('Test Execution Timeline')
    ax.set_yscale('log')

    buf = io.BytesIO()
    plt.savefig(buf, format='png', bbox_inches='tight')
    charts['timeline'] = base64.b64encode(buf.getvalue()).decode()
    plt.close()

    return charts

def _analyze_failures(self, results: list) -> dict:
    """ 失败模式分析"""
    failures = [r for r in results if r.get('status') in ('FAIL', 'ERROR')]

    nrc_distribution = {}
    for f in failures:
        nrc = f.get('response', {}).get('nrc', 'UNKNOWN')
```

```
            if nrc not in nrc_distribution:
                nrc_distribution[nrc] = {'count': 0, 'cases': []}
            nrc_distribution[nrc]['count'] += 1
            nrc_distribution[nrc]['cases'].append(f.get('id', 'unknown'))

        # 按服务分类
        service_failures = {}
        for f in failures:
            sid = f.get('request', [None])[0] if isinstance(f.get('request'), list) else None
            service_name = self._sid_to_name(sid) if sid else 'UNKNOWN'
            service_failures.setdefault(service_name, []).append(f.get('id'))

        return {
            'nrc_distribution': nrc_distribution,
            'service_failures': service_failures,
            'total_failures': len(failures)
        }

    def _sid_to_name(self, sid: int) -> str:
        """SID 转换为服务名称"""
        names = {
            0x10: 'SessionControl', 0x11: 'ECUReset', 0x14: 'ClearDTC',
            0x19: 'ReadDTC', 0x22: 'ReadDataByID', 0x27: 'SecurityAccess',
            0x2E: 'WriteDataByID', 0x31: 'RoutineControl', 0x34: 'RequestDownload',
            0x36: 'TransferData', 0x37: 'RequestTransferExit', 0x3E: 'TesterPresent'
        }
        return names.get(sid, f'Unknown(0x{sid:02X})')

    def _enrich_results(self, results: list) -> list:
        """ 丰富结果数据用于模板渲染"""
        import json
        enriched = []
        for r in results:
            er = dict(r)
            er['service_name'] = self._sid_to_name(
                r.get('request', [None])[0] if isinstance(r.get('request'), list) else None
            )
            er['details_json'] = json.dumps(r, indent=2, default=str)
            enriched.append(er)
        return enriched
```

4. KWP2000 诊断协议自动化测试

4.1 KWP2000 协议特性与配置

4.1.1 物理层与数据链路层参数（K 线/L 线） KWP2000（Keyword Protocol 2000，ISO 14230）作为 UDS 的前身协议，在物理层和数据链路层具有显著差异。其物理层基于 **K 线（单线通信，ISO 9141-2）**或 **L 线（双线增强）**，采用异步串行传输，典型波特率为 **10400bps 或 9600bps**，数据格式为 **8N1**（8 数据位、无校验、1 停止位）(CSDN 文库) 。与 CAN-based UDS 相比，KWP2000 的实现成本更低，但抗干扰能力和传输速率较弱，主要应用于车身电子、舒适系统等低速场景。

TSMaster 通过 **TSCANLIN 系列硬件**支持 KWP2000 物理层，配置参数包括：

参数	典型值	说明
波特率	10400 / 9600 / 自定义	需与 ECU 严格匹配
数据位	8	固定
停止位	1	固定
校验	无	固定
空闲电平	高电平（12V）	K 线特性
逻辑 0	低电平（0V）	接地

4.1.2 慢速初始化与快速初始化机制 KWP2000 定义了两种链路建立机制，这是与 UDS 的关键差异：

初始化方式	时序特征	建立时间	应用场景
慢速初始化（5-baud init）	地址字节以 5bps 发送，ECU 以 10.4Kbps 响应	~2 秒	兼容早期 ISO 9141-2 系统
快速初始化（fast init）	25ms 低电平 + 25ms 高电平启动序列	~100ms	现代 KWP2000 系统首选

慢速初始化详细流程：

1. 诊断仪发送 **5bps 的地址字节**（通常为 0x33 或 ECU 特定地址）
2. 等待 ECU 以 **10.4Kbps** 发送**关键字节（Key Bytes）**
3. 关键字节包含格式标识和波特率信息
4. 双方确认后，切换到正常工作波特率

快速初始化流程：

1. 总线保持 **25ms 低电平**（显性）
2. 总线保持 **25ms 高电平**（隐性）
3. ECU 检测到边沿后准备接收
4. 诊断仪立即发送 **10.4Kbps 的启动报文**

TSMaster 的 KWP2000 配置界面支持初始化方式选择，COM API 提供 `SetInitializationMode()` 方法动态切换。自动化测试应覆盖两种机制，验证 ECU 的兼容性和响应行为。

4.1.3 服务标识符与 UDS 的差异映射 KWP2000 与 UDS 的服务标识符存在映射关系，但功能集不完全等价：

KWP2000 SID	服务名称	UDS 等效 SID	差异说明
0x81	StartCommunication	无直接等效	KWP2000 特有，建立诊断会话
0x82	StopCommunication	无直接等效	KWP2000 特有，终止会话
0x83	AccessTimingParameter	无直接等效	读写通信时序参数
0x10	StartDiagnosticSession	0x10	子功能定义不同
0x11	StopDiagnosticSession	无直接等效	UDS 通过会话超时管理
0x14	ClearDiagnosticInformation	0x14	功能等效

Table 17 – continued

KWP2000 SID	服务名称	UDS 等效 SID	差异说明
0x18	ReadDiagnosticTroubleCodes	0x19	UDS 功能更细分
0x21	ReadDataByLocalIdentifier	0x22	**1 字节本地 ID vs 2 字节 DID**
0x22	ReadDataByCommonIdentifier	无直接等效	读取公共数据
0x27	SecurityAccess	0x27	机制类似，Seed/Key 长度可能不同
0x2C	DynamicallyDefineLocalIdentifier	无直接等效	动态定义数据标识
0x30	InputOutputControlByLocalIdentifier	0x2F	功能等效，ID 长度差异
0x31	StartRoutineByLocalIdentifier	0x31	子功能简化
0x32	StopRoutineByLocalIdentifier	0x31	UDS 通过 routineType 区分
0x33	RequestRoutineResultsByLocalIdentifier	0x31	UDS 集成到 RoutineControl
0x34	RequestDownload	0x34	功能等效
0x35	RequestUpload	0x35	功能等效
0x36	TransferData	0x36	功能等效
0x37	RequestTransferExit	0x37	功能等效
0x3B	WriteDataByLocalIdentifier	0x2E	**1 字节本地 ID vs 2 字节 DID**

关键差异：KWP2000 使用 **1 字节本地标识符（Local Identifier）**，地址空间仅 256 个；UDS 使用 **2 字节数据标识符（DID）**，地址空间 65536 个。自动化测试脚本需要处理这一差异，特别是在跨协议测试系统中。

4.2 KWP2000 服务测试实现

4.2.1 启动通信与停止通信（$81/$82）

KWP2000 会话生命周期管理测试：

```python
class KWP2000SessionTest:
    def __init__(self, kwp_driver):
        self.driver = kwp_driver  # TSMaster KWP2000 驱动接口

    def test_session_lifecycle(self):
        """ 测试完整会话生命周期"""
        results = []

        # $81 StartCommunication
        start_result = self._start_communication()
        results.append({'phase': 'start', **start_result})

        if not start_result['success']:
            return results  # 无法建立通信

        # 执行诊断操作
        diag_results = self._execute_diagnostic_sequence()
        results.extend(diag_results)
```

```python
        # $82 StopCommunication
        stop_result = self._stop_communication()
        results.append({'phase': 'stop', **stop_result})

        return results

    def _start_communication(self, init_mode='fast'):
        """ 启动通信，支持慢速/快速初始化"""
        # 配置初始化方式
        self.driver.SetInitializationMode(0 if init_mode == 'slow' else 1)

        # 发送 $81 请求
        # 格式: [0x81, 格式字节, 目标地址, 源地址]
        request = [0x81, 0x01, 0x33, 0xF1]  # 典型参数

        response = self.driver.ExecuteService(request, timeout=3000)

        if response['success']:
            # 解析关键字节
            key_bytes = response.get('data', [])
            return {
                'success': True,
                'init_mode': init_mode,
                'key_bytes': [f'0x{b:02X}' for b in key_bytes],
                'negotiated_baudrate': self._parse_baudrate(key_bytes)
            }

        return {
            'success': False,
            'error': response.get('error'),
            'nrc': response.get('nrc')
        }

    def _stop_communication(self):
        """ 停止通信"""
        # $82 请求格式简单
        request = [0x82]

        response = self.driver.ExecuteService(request, timeout=1000)

        return {
            'success': response['success'],
            'note': 'ECU 应返回默认状态，总线静默'
        }
```

4.2.2 访问时序参数（$83）

KWP2000 时序参数读写测试：

```python
def test_timing_parameters(self):
    """ 测试 $83 访问时序参数"""
    results = []
```

```python
    # 读取当前时序参数（子功能 0x01）
    read_request = [0x83, 0x01]  # readLimits
    read_response = self.driver.ExecuteService(read_request)

    if read_response['success']:
        p1_min, p1_max, p2_min, p2_max, p3_min, p3_max, p4_min = \
            self._parse_timing_limits(read_response['data'])

        results.append({
            'operation': 'read_limits',
            'p1_range_ms': [p1_min, p1_max],  # ECU 响应时间
            'p2_range_ms': [p2_min, p2_max],  # ECU 到诊断仪间隔
            'p3_range_ms': [p3_min, p3_max],  # 诊断仪到 ECU 间隔
            'p4_min_ms': p4_min               # 诊断仪连续发送最小间隔
        })

    # 设置新时序参数（子功能 0x03，需在默认会话）
    new_p2 = 50  # 缩短 P2 以提高吞吐量
    set_request = [0x83, 0x03, new_p2]  # setValues
    set_response = self.driver.ExecuteService(set_request)

    results.append({
        'operation': 'set_p2',
        'requested_p2_ms': new_p2,
        'success': set_response['success'],
        'actual_p2_ms': self._verify_p2() if set_response['success'] else None
    })

    # 重置为默认值（子功能 0x04）
    reset_response = self.driver.ExecuteService([0x83, 0x04])
    results.append({
        'operation': 'reset_to_default',
        'success': reset_response['success']
    })

    return results
```

4.2.3 安全访问与数据读写服务适配 KWP2000 安全访问与数据服务适配层:

```python
class KWP2000Adapter:
    """
    KWP2000 到 UDS 的适配层
    统一接口，隐藏协议差异
    """

    def __init__(self, kwp_driver):
        self.kwp = kwp_driver
        self.session_active = False

    # ========== 会话控制适配 ==========
```

```python
    def diagnostic_session_control(self, session_type: int) -> dict:
        """
        适配 UDS $10 到 KWP2000 $10

        session_type 映射:
        0x01 (Default)  -> KWP2000 0x01 (default)
        0x03 (Extended) -> KWP2000 0x85 (extended) 或 0x02 (programming)
        0x02 (Programming) -> KWP2000 0x02 (programming)
        """
        kwp_session_map = {
            0x01: 0x01,  # default
            0x03: 0x85,  # extended diagnostic mode
            0x02: 0x02,  # programming
        }

        kwp_type = kwp_session_map.get(session_type, 0x01)
        request = [0x10, kwp_type]

        # KWP2000 可能需要额外的通信参数
        if session_type == 0x02:  # programming
            request.extend([0x01, 0x00])  # 波特率标识等

        response = self.kwp.ExecuteService(request)

        return {
            'success': response['success'],
            'session_type': session_type,
            'kwp_type': kwp_type,
            'data': response.get('data', [])
        }

    # ========== 数据读写适配 ==========

    def read_data_by_identifier(self, identifier: int) -> dict:
        """
        适配 UDS $22 (2-byte DID) 到 KWP2000 $21 (1-byte local ID)

        对于大于 0xFF 的 DID，需要:
        1. 使用 $2C 动态定义本地标识符
        2. 或通过 $22 映射（如 ECU 支持）
        """
        if identifier <= 0xFF:
            # 直接使用本地标识符
            request = [0x21, identifier]
            response = self.kwp.ExecuteService(request)

            return {
                'success': response['success'],
                'identifier': identifier,
                'data': response.get('data', [])[1:] if response['success'] else [],
                'method': 'direct_local_id'
```

```
        }
    else:
        # 需要动态定义或使用公共标识符
        return self._read_extended_identifier(identifier)

def _read_extended_identifier(self, did: int) -> dict:
    """ 处理超过 1 字节的标识符"""
    # 方法 1: 尝试使用 $2C 动态定义
    local_id = 0xF0  # 保留的动态 ID 范围

    define_request = [
        0x2C, 0x01,  # dynamicallyDefineByIdentifier
        local_id,    # 分配的本地 ID
        0x02,        # DID 长度 2 字节
        *did.to_bytes(2, 'big')
    ]

    define_response = self.kwp.ExecuteService(define_request)

    if define_response['success']:
        # 使用动态定义的本地 ID 读取
        read_response = self.kwp.ExecuteService([0x21, local_id])

        # 清理动态定义
        self.kwp.ExecuteService([0x2C, 0x03, local_id])  # clearDynamicDefinition

        return {
            'success': read_response['success'],
            'original_did': did,
            'local_id': local_id,
            'data': read_response.get('data', [])[1:] if read_response['success'] else []
        }

    # 方法 2: 查找预定义的公共标识符映射
    return {
        'success': False,
        'error': f'Extended DID 0x{did:04X} not supported on KWP2000',
        'suggestion': 'Use UDS protocol or predefined mapping table'
    }

# ========== 安全访问适配 ==========

def security_access(self, level: int, seed_key_func: callable) -> dict:
    """
    KWP2000 $27 安全访问，与 UDS 机制类似
    但 Seed/Key 长度和算法可能不同
    """
    # 请求 Seed
    request_seed = [0x27, level * 2 - 1]
    seed_response = self.kwp.ExecuteService(request_seed)
```

```
        if not seed_response['success']:
            return {'success': False, 'phase': 'request_seed', 'error': seed_response.get('nrc')}

        seed = seed_response.get('data', [])[1:]  # 跳过子功能字节

        # 使用提供的算法计算 Key
        key = seed_key_func(seed, level)

        # 发送 Key
        send_key = [0x27, level * 2] + list(key)
        key_response = self.kwp.ExecuteService(send_key)

        return {
            'success': key_response['success'],
            'level': level,
            'seed_length': len(seed),
            'key_length': len(key),
            'phase': 'complete' if key_response['success'] else 'send_key'
        }
```

5. ECU 功能测试与集成验证

5.1 功能测试用例设计

5.1.1 基于需求的测试场景定义 ECU 功能测试用例设计遵循 **V 模型**和**需求追溯原则**，确保每个功能需求都有对应的测试验证。测试场景来源包括：

需求来源	典型内容	测试重点
系统需求规格（SyRS）	功能定义、性能指标、接口规范	黑盒功能验证、边界测试
软件需求规格（SRS）	算法逻辑、状态机、数据流	白盒覆盖、路径测试
诊断需求规范	UDS/KWP2000 服务支持、DTC 定义	协议一致性、故障注入
网络安全需求（ISO/SAE 21434）	安全启动、认证加密、入侵检测	渗透测试、模糊测试
功能安全需求（ISO 26262）	ASIL 等级、安全机制、FTTI	故障注入、安全机制验证

测试场景模板：

```
test_case:
  id: TC_PWR_001
  title: 发动机启动扭矩控制验证
  requirement_refs: [SYRS_PWR_010, SRS_ENG_045]
  asil_level: ASIL_D

  precondition:
    vehicle_state: stationary
    battery_voltage: [11.5, 14.5]  # V
    coolant_temp: [-40, 120]       # °C

  stimulus:
    driver_request: start_engine
```

```
    brake_pedal: pressed
    gear_position: P_or_N

  expected_behavior:
    starter_engagement_delay_ms: [0, 200]
    cranking_rpm: [150, 300]
    fuel_injection_start: at_100_rpm
    successful_start_indicator: engine_speed_stable_above_500_rpm_within_2s

  pass_criteria:
    all_timing_requirements_met: true
    no_DTC_set: true
    safety_monitor_silent: true
```

5.1.2 边界条件与异常工况模拟 边界测试和故障注入策略:

测试类别	具体方法	TSMaster 实现
信号边界值	最小值、最大值、临界值、非法值	信号生成器设置精确边界序列
时序边界	最小周期、最大延迟、抖动容限	周期报文动态调整、延迟注入
通信故障	帧丢失、重复、乱序、损坏	报文过滤/阻断、错误帧注入
电源扰动	欠压、过压、瞬断、纹波	可编程电源控制（需外部设备）
温度极端	高温、低温、温度变化率	环境舱控制（需外部设备）
EMC 干扰	传导发射、辐射抗扰	暗室测试（专用设施）

TSMaster 的 **RBS 残余总线仿真**和**小程序编程**支持灵活的故障注入:

```python
# 示例: 通信延迟注入
def inject_communication_delay(channel, target_id, delay_ms):
    """ 在指定通道对目标 ID 的报文注入延迟"""

    # 设置接收回调，延迟后转发
    original_handler = channel.GetReceiveCallback()

    def delayed_forward(msg):
        if msg.ID == target_id:
            # 延迟转发
            def forward_after_delay():
                time.sleep(delay_ms / 1000.0)
                # 修改时间戳后转发
                msg.Timestamp += delay_ms * 1000  # μs
                channel.Transmit(msg)

            threading.Thread(target=forward_after_delay).start()
            return  # 不立即处理

    channel.SetReceiveCallback(delayed_forward)

    # 恢复函数
    def restore():
```

```
        channel.SetReceiveCallback(original_handler)

    return restore
```

5.1.3 时序相关功能测试 时序测试的关键技术：

技术	精度	应用场景	TSMaster 支持
硬件时间戳	1 s	网络延迟测量、同步验证	全系列硬件支持
PTP 时间同步	<1 s	多设备分布式同步	高端型号支持
循环触发	1ms（软件）/100 s（硬件）	周期性信号激励	小程序/图形编程
事件触发	<1ms	响应时间测试	系统变量 + 回调
逻辑分析仪	ns 级	物理层信号分析	需外部设备

5.2 测试系统集成

5.2.1 HIL（硬件在环）测试架构 TSMaster HIL 集成架构：

```
┌──────────────────────────────────────────────────────────┐
│                      HIL 测试系统架构                      │
├──────────────────────────────────────────────────────────┤
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────┐ │
│  │  被测 ECU    │◄──►│  TSMaster    │◄──►│  仿真模型    │ │
│  │  (真实硬件)  │    │  (接口枢纽)  │    │(Simulink/    │ │
│  │              │    │              │    │ CarSim/      │ │
│  │  CAN/LIN/    │    │  报文路由    │    │ 自定义)      │ │
│  │  模拟 IO/    │    │  信号转换    │    │              │ │
│  │  数字 IO     │    │  故障注入    │    │  车辆动力学  │ │
│  │              │    │  数据记录    │    │  传感器模型  │ │
│  └──────────────┘    └──────┬───────┘    │  执行器模型  │ │
│                             │            └──────────────┘ │
│                        ┌────┴────┐                       │
│                        │ 自动化测试引擎                   │
│                        │ (Python/C#/TestStand)           │
│                        └─────────┘                       │
└──────────────────────────────────────────────────────────┘
```

TSMaster 与 MATLAB/Simulink 联合仿真配置（上海同星智能）：

1. 在 Simulink 中安装 TSMaster S-Function 模块
2. 配置双向信号接口：Simulink→TSMaster（激励信号）、TSMaster→Simulink（ECU 响应）
3. 设置同步机制：基于步长的软同步或基于硬件触发的硬同步
4. 联合调试：Simulink 实时观测 TSMaster 总线数据，TSMaster 面板显示 Simulink 模型变量

5.2.2 信号激励与采集同步 多源信号同步策略：

同步级别	实现方式	精度	配置复杂度
软件同步	统一软件时间戳	~10ms	低
网络同步	IEEE 1588 PTP	<1ms	中

Table 21 – continued

同步级别	实现方式	精度	配置复杂度
硬件触发	外部触发输入/输出	<100 s	中
时钟同步	共享 10MHz 参考时钟	<1 s	高

TSMaster 硬件触发配置:

```
# 配置通道 1 为触发主设备
master_ch = network.Channels(1)
master_ch.TriggerMode = 1  # Trigger Master
master_ch.TriggerOutput = True  # 输出触发信号

# 配置通道 2-4 为触发从设备
for i in [2, 3, 4]:
    ch = network.Channels(i)
    ch.TriggerMode = 2  # Trigger Slave
    ch.TriggerInputSource = 1  # 来自通道 1

# 启动同步采集
network.StartSynchronizedAcquisition()
```

5.2.3 自动化测试序列编排 测试序列编排的层次结构:

层次	名称	功能	实现工具
L1	**测试步骤（Step）**	最小可执行单元，单次诊断服务或信号操作	TSM 诊断控制台/小程序
L2	**测试用例（Case）**	完整的功能验证场景，多步骤组合	Python unittest/pytest
L3	**测试套件（Suite）**	相关用例的集合，如”启动系统测试”	测试框架组织
L4	**测试活动（Campaign）**	完整验证周期，多套件组合执行	CI/CD 流水线
L5	**回归测试（Regression）**	自动化周期性执行，版本对比	Jenkins/GitLab CI

TSMaster 图形化编程与代码编程的协同:

- **快速原型**: 图形化编程模块拖拽搭建，验证测试逻辑
- **生产固化**: 转换为 Python/C 小程序，纳入版本控制
- **混合执行**: 图形化流程调用代码小程序，代码中嵌入图形化面板

5.3 测试执行与结果判定

5.3.1 实时信号监测与阈值判定 多维度判定策略:

判定类型	描述	实现方式	适用场景
瞬时值判定	单点数值范围检查	`check_verdict` 即时调用	稳态验证

Table 23 – continued

判定类型	描述	实现方式	适用场景
时段统计判定	时间窗口内的均值/方差/极值	Signal Checker 时段配置	动态过程
趋势判定	变化率、单调性、收敛性	自定义算法计算	启动/过渡过程
相关性判定	多信号间的数学关系	表达式引擎或自定义代码	复杂功能验证
时序判定	事件先后顺序、时间间隔	状态机或时间戳分析	状态转换验证

TSMaster Signal Checker 配置示例:

```
# 配置时段判定：发动机启动过程转速验证
checker = automation.AddSignalChecker("EngineStart_RPM")

checker.SignalPath = "Powertrain/EngineData/EngineSpeed"
checker.CheckType = "TIME_WINDOW"  # 时段判定
checker.StartCondition = "IgnitionSwitch == 'START'"  # 触发条件
checker.DurationMs = 2000  # 2 秒监测窗口

# 统计类型
checker.StatType = "ALL"  # 记录所有统计量
checker.ExpectedMin = 0
checker.ExpectedMax = 6000  # 转速上限

# 判定规则
checker.PassCondition = "MIN >= 0 AND MAX <= 6000 AND RISE_TIME_TO(500) <= 500"

# 失败动作
checker.OnFail = "LOG_AND_CONTINUE"  # 记录并继续
```

5.3.2 测试 verdict 自动判定机制 Verdict 聚合规则:

层级	聚合方式	说明
检查点（Check）	单一条件判定	PASS/FAIL/INCONCLUSIVE
步骤（Step）	检查点组合	关键检查点 FAIL 则步骤 FAIL
用例（Case）	步骤组合	任意步骤 FAIL 则用例 FAIL
套件（Suite）	用例统计	通过率阈值判定
活动（Campaign）	多维度报告	综合质量评分

Verdict 状态机:

```
INCONCLUSIVE ──▶ PASS  (所有检查通过)
        │
        ├──▶ FAIL  (关键检查失败)
        │
        └──▶ ERROR (执行异常，需人工判定)
              │
              ▼
        RECOVERABLE ──▶ 重试或替代路径
              │
```

```
        NON-RECOVERABLE ──▶ 终止并告警
```

5.3.3 测试报告自动生成与追溯 报告内容结构与追溯性:

章节	内容	追溯目标
执行摘要	时间、环境、版本、执行人、总体结果	测试管理
需求覆盖矩阵	需求 ID 测试用例 执行结果	需求管理
详细结果	每个用例的步骤、数据、判定、截图	问题分析
趋势分析	历史执行对比、缺陷密度、稳定性	质量改进
附件	原始数据记录 (BLF/ASC)、配置文件	复现验证

TSMaster 报告生成集成:

```python
def generate_comprehensive_report(test_results, output_dir):
    """ 生成综合测试报告包"""

    # 1. TSMaster 原生报告
    tsm_report = app.Automation.GenerateReport(
        format='HTML',
        includeRawData=True,
        includeScreenshots=True
    )

    # 2. 自定义 Python 分析报告
    analysis = analyze_test_trends(test_results)

    # 3. JUnit XML (CI 集成)
    junit_xml = convert_to_junit(test_results)

    # 4. 需求追溯报告
    coverage = generate_requirement_coverage(test_results)

    # 打包输出
    report_package = {
        'timestamp': datetime.now().isoformat(),
        'tsm_report': tsm_report,
        'analysis': analysis,
        'junit': junit_xml,
        'coverage': coverage,
        'raw_data': collect_raw_data()
    }

    # 上传测试管理系统 (如 Polarion、Jama)
    if CONFIG['upload_to_tms']:
        upload_to_test_management(report_package)

    return report_package
```

6. 高级功能与最佳实践

6.1 多 ECU 并行测试

6.1.1 多通道独立配置 TSMaster **64 通道架构**支持复杂的多 ECU 测试场景 (上海同星智能) 。通道独立配置策略:

场景	通道分配	同步需求
单 ECU 多网络	1 个 ECU 连接 CAN+LIN+Ethernet	网络间路由延迟测量
同型 ECU 批量	N 个相同 ECU，各占用 1 通道	并行执行相同测试序列
异型 ECU 集成	动力 ECU+ 车身 ECU+ 信息娱乐 ECU	跨 ECU 功能时序验证
网关 ECU 测试	多网络接口 + 仿真节点	协议转换正确性验证

多 ECU 配置代码框架:

```python
class MultiECUTestRig:
    def __init__(self, app):
        self.app = app
        self.ecus = {}  # ECU 名称到通道配置的映射

    def register_ecu(self, name: str, channel_config: dict):
        """ 注册 ECU 及其通道配置"""
        self.ecus[name] = {
            'channels': {},  # 通道 ID 到对象映射
            'config': channel_config,
            'diagnostic': None,  # 诊断接口
            'state': 'IDLE'
        }

    def initialize_all(self):
        """ 初始化所有 ECU 连接"""
        for name, ecu in self.ecus.items():
            for ch_id, ch_cfg in ecu['config']['channels'].items():
                ch = self._configure_channel(ch_id, ch_cfg)
                ecu['channels'][ch_id] = ch

            # 创建诊断接口
            ecu['diagnostic'] = self._create_diagnostic_interface(
                name,
                ecu['config']['diagnostic']
            )

    def execute_parallel(self, test_function, params_per_ecu: dict) -> dict:
        """ 在所有 ECU 上并行执行测试"""
        with ThreadPoolExecutor(max_workers=len(self.ecus)) as executor:
            futures = {
                executor.submit(
                    test_function,
                    ecu['diagnostic'],
                    params_per_ecu.get(name, {})
```

```
            ): name
            for name, ecu in self.ecus.items()
        }

        results = {}
        for future in as_completed(futures):
            name = futures[future]
            try:
                results[name] = future.result()
            except Exception as e:
                results[name] = {'error': str(e)}

        return results
```

6.1.2 测试任务调度与资源管理 动态任务调度策略：

策略	描述	适用场景
静态分配	预定义 ECU-测试映射	产线固定工位
动态队列	测试任务入队，空闲 ECU 抢占执行	研发弹性测试
优先级调度	高优先级任务抢占低优先级	紧急验证需求
依赖调度	按 ECU 间依赖关系排序执行	集成测试序列

资源冲突处理：

- **通道争用**：通道池管理，请求-等待-超时机制
- **电源限制**：可编程电源功率管理，分时上电
- **散热约束**：温度监控，超温降频或暂停

6.2 测试数据管理与分析

6.2.1 总线数据记录与回放（BLF/ASC/CSV 格式） TSMaster 支持格式特性对比：

格式	扩展名	特点	适用场景
BLF	.blf	二进制，紧凑高效，Vector 标准	长时间记录，原始存档
ASC	.asc	文本可读，版本控制友好	人工审查，差异比较
CSV	.csv	表格结构，Excel 兼容	快速分析，报表生成
MAT	.mat	MATLAB 原生，含信号解析	算法开发，离线仿真
MF4	.mf4	ASAM 标准，多源数据融合	企业数据管理

记录策略配置：

```
# 条件触发记录
recorder = app.CreateDataRecorder()
```

```
# 持续记录（全量）
recorder.Mode = "CONTINUOUS"

# 触发记录（特定事件前后）
recorder.Mode = "TRIGGER"
recorder.PreTriggerTime = 5000   # 触发前 5 秒
recorder.PostTriggerTime = 10000 # 触发后 10 秒
recorder.TriggerCondition = "DiagnosticNRC == 0x78 OR BusLoad > 80"

# 分段记录（按大小或时间）
recorder.Mode = "SEGMENTED"
recorder.SegmentSizeMB = 100
recorder.SegmentDurationMin = 30
```

6.2.2 信号趋势分析与统计评估 离线分析算法集成：

```
import numpy as np
from scipy import signal, stats

class SignalAnalyzer:
    def __init__(self, recorded_data):
        self.data = recorded_data  # 从 BLF/ASC 加载

    def analyze_signal(self, signal_path: str) -> dict:
        """ 综合信号分析"""
        timestamps, values = self._extract_signal(signal_path)

        return {
            'basic_stats': self._basic_statistics(values),
            'trend': self._trend_analysis(timestamps, values),
            'frequency': self._frequency_analysis(values),
            'anomalies': self._anomaly_detection(timestamps, values),
            'correlations': self._correlation_analysis(signal_path)
        }

    def _basic_statistics(self, values):
        """ 基础统计"""
        return {
            'count': len(values),
            'mean': np.mean(values),
            'std': np.std(values),
            'min': np.min(values),
            'max': np.max(values),
            'percentiles': {
                'p50': np.percentile(values, 50),
                'p95': np.percentile(values, 95),
                'p99': np.percentile(values, 99)
            }
        }

    def _trend_analysis(self, timestamps, values):
```

```python
        """ 趋势分析：线性回归、变化率"""
        # 线性趋势
        slope, intercept, r_value, p_value, std_err = stats.linregress(
            timestamps, values
        )

        # 变化率（移动窗口）
        window_size = min(100, len(values) // 10)
        if window_size > 1:
            rates = np.gradient(values, timestamps)
            moving_avg_rate = np.convolve(rates, np.ones(window_size)/window_size, mode='valid')
        else:
            moving_avg_rate = []

        return {
            'linear_trend_slope': slope,
            'linear_trend_r2': r_value**2,
            'max_positive_rate': np.max(rates) if len(rates) > 0 else None,
            'max_negative_rate': np.min(rates) if len(rates) > 0 else None,
            'rate_stability': np.std(moving_avg_rate) if len(moving_avg_rate) > 0 else None
        }

    def _frequency_analysis(self, values, fs=1000):
        """ 频域分析"""
        # FFT
        fft_vals = np.fft.fft(values)
        freqs = np.fft.fftfreq(len(values), 1/fs)

        # 功率谱密度
        psd = np.abs(fft_vals)**2

        # 主频率成分
        dominant_idx = np.argmax(psd[1:len(psd)//2]) + 1  # 排除 DC

        return {
            'dominant_frequency': freqs[dominant_idx],
            'dominant_power': psd[dominant_idx],
            'frequency_bins': freqs[:len(freqs)//2].tolist(),
            'psd': psd[:len(psd)//2].tolist()
        }

    def _anomaly_detection(self, timestamps, values, threshold=3):
        """ 基于 Z-score 的异常检测"""
        z_scores = np.abs(stats.zscore(values))
        anomaly_indices = np.where(z_scores > threshold)[0]

        return {
            'anomaly_count': len(anomaly_indices),
            'anomaly_ratio': len(anomaly_indices) / len(values),
            'anomaly_events': [
                {
```

```
                    'index': int(idx),
                    'timestamp': timestamps[idx],
                    'value': values[idx],
                    'z_score': z_scores[idx]
                }
                for idx in anomaly_indices[:10]  # 前 10 个
            ]
        }
```

6.2.3 测试数据库集成 企业级测试数据管理架构:

```
┌──────────────────────────────────────────┐
│            测试数据管理系统               │
├──────────────────────────────────────────┤
│  数据采集层  │ TSMaster → ETL → 数据湖   │
├──────────────┼──────────────────────────┤
│  数据存储层  │ 时序数据库 (InfluxDB)      │
│              │ 关系数据库 (PostgreSQL)    │
│              │ 对象存储 (S3/MinIO)        │
├──────────────┼──────────────────────────┤
│  数据分析层  │ Jupyter/Spark/自定义算法   │
├──────────────┼──────────────────────────┤
│  应用展示层  │ Grafana/自定义报表/BI 工具 │
├──────────────┼──────────────────────────┤
│  集成接口层  │ REST API/ODBC/消息队列     │
└──────────────────────────────────────────┘
```

TSMaster 数据导出配置:

```python
# 实时流式导出到 InfluxDB
influx_exporter = app.CreateDataExporter('INFLUXDB')
influx_exporter.Configure(
    url='http://influxdb:8086',
    database='tsmaster_tests',
    measurement='can_signals',
    tags=['project', 'test_run_id', 'ecu_version']
)

# 批量导出到数据仓库
def nightly_sync_to_warehouse():
    """ 夜间同步当日测试数据"""
    today = datetime.now().strftime('%Y-%m-%d')

    # 查询当日所有测试记录
    test_runs = query_local_database(f"SELECT * FROM test_runs WHERE date='{today}'")

    for run in test_runs:
        # 上传关联的 BLF 文件
        upload_to_s3(run['blf_path'], f"s3://test-data/{today}/")

        # 同步结构化结果
```

```
        insert_to_warehouse('test_results', run['summary'])

        # 更新测试管理系统状态
        update_tms_status(run['tms_id'], 'ARCHIVED')
```

6.3 系统可靠性与错误处理

6.3.1 通信中断恢复机制 分层恢复策略:

中断层级	检测方式	恢复动作	恢复时间
单帧丢失	序列号不连续	自动重传（流控帧请求）	<10ms
临时总线关闭	错误计数器超限	自动总线恢复（128 次空闲）	~100ms
物理层断开	连接状态检测	重新初始化通道	1-5s
ECU 复位	诊断无响应	完整会话重建	5-30s
TSMaster 崩溃	看门狗监控	进程重启，状态恢复	10-60s

自动恢复实现:

```
class ResilientCommunicationLayer:
    def __init__(self, app, max_retries=3):
        self.app = app
        self.max_retries = max_retries
        self.state = 'CONNECTED'
        self.recovery_procedures = {
            'FRAME_LOSS': self._recover_frame_loss,
            'BUS_OFF': self._recover_bus_off,
            'PHYSICAL_DISCONNECT': self._recover_physical,
            'ECU_RESET': self._recover_ecu_reset,
            'APPLICATION_CRASH': self._recover_application
        }

    def execute_with_resilience(self, operation, *args, **kwargs):
        """ 带恢复机制的操作执行"""
        for attempt in range(self.max_retries):
            try:
                # 状态检查
                if self.state != 'CONNECTED':
                    self._attempt_recovery()

                # 执行操作
                result = operation(*args, **kwargs)
                return {'success': True, 'result': result, 'attempts': attempt + 1}

            except CommunicationException as e:
                error_type = self._classify_error(e)
                self._log_error(f"Attempt {attempt+1} failed: {error_type}")

                if attempt < self.max_retries - 1:
                    recovered = self._execute_recovery(error_type)
                    if not recovered:
```

```
                        break  # 恢复失败，终止重试
                else:
                    break  # 重试耗尽

        return {'success': False, 'error': error_type, 'attempts': attempt + 1}

    def _recover_ecu_reset(self):
        """ECU 复位恢复：完整会话重建"""
        recovery_steps = [
            ('等待 ECU 启动', lambda: time.sleep(2)),
            ('检测 ECU 在线', self._ping_ecu),
            ('重建诊断会话', self._rebuild_session),
            ('恢复安全访问', self._re_unlock_security),
            ('恢复应用状态', self._restore_application_state)
        ]

        for name, action in recovery_steps:
            if not action():
                self._log_error(f"Recovery failed at: {name}")
                return False

        self.state = 'CONNECTED'
        return True
```

6.3.2 诊断超时异常处理 智能超时管理：

```
class AdaptiveTimeoutManager:
    """ 自适应超时管理器"""

    DEFAULT_P2 = 50      # ms
    DEFAULT_P2STAR = 5000  # ms
    ADAPTATION_WINDOW = 10  # 样本数

    def __init__(self):
        self.response_times = []  # 历史响应时间
        self.current_p2 = self.DEFAULT_P2
        self.current_p2star = self.DEFAULT_P2STAR

    def record_response_time(self, elapsed_ms: float, was_pending: bool):
        """ 记录响应时间，自适应调整"""
        self.response_times.append({
            'elapsed': elapsed_ms,
            'pending': was_pending,
            'timestamp': time.time()
        })

        # 保持窗口大小
        if len(self.response_times) > self.ADAPTATION_WINDOW:
            self.response_times.pop(0)

        # 自适应调整（仅基于非 pending 响应）
```

```
        normal_responses = [r for r in self.response_times if not r['pending']]
        if len(normal_responses) >= 5:
            avg_time = sum(r['elapsed'] for r in normal_responses) / len(normal_responses)
            std_time = (sum((r['elapsed'] - avg_time)**2 for r in normal_responses) /
             ↪  len(normal_responses)) ** 0.5

            # P2 设置为均值 +3 倍标准差，确保 99.7% 覆盖
            self.current_p2 = min(max(avg_time + 3*std_time, 25), 200)  # 边界限制

            # P2* 保持相对固定，或基于最大观测值
            max_observed = max(r['elapsed'] for r in self.response_times)
            self.current_p2star = max(max_observed * 2, self.DEFAULT_P2STAR)

    def get_timeouts(self, service_sid: int = None) -> tuple:
        """ 获取当前推荐的超时设置"""
        # 特定服务可配置特殊超时
        service_overrides = {
            0x34: (100, 30000),   # RequestDownload: 擦除可能耗时
            0x31: (200, 60000),   # RoutineControl: 例程执行
        }

        return service_overrides.get(service_sid, (self.current_p2, self.current_p2star))
```

6.3.3 测试环境自诊断 预测试健康检查清单:

检查项	检查方法	通过标准	失败处理
TSMaster 许可证	API 查询	所需功能模块已授权	中止测试，联系管理员
硬件连接状态	通道状态查询	所有配置通道已连接	检查线缆，重试连接
总线终端电阻	物理层诊断	电阻值在规范范围	提示检查终端电阻
通信质量基线	发送测试帧	错误帧率为 0	检查网络负载，隔离干扰源
ECU 响应性	诊断会话请求	正常响应时间 $<$P2	检查 ECU 电源，尝试复位
时间同步精度	多通道同步测试	通道间偏差 $<$100 s	启用硬件同步，检查时钟源
存储空间	磁盘查询	可用空间 $>$10GB	清理历史数据，扩展存储
系统资源	CPU/内存监控	CPU$<$80%, 内存 $<$90%	关闭非必要进程，优化配置

自诊断代码集成:

```
def pre_test_health_check(app, config: dict) -> dict:
    """ 执行完整的预测试健康检查"""

    checks = []

    # 1. 许可证检查
    license_check = _check_license(app, config['required_features'])
    checks.append(license_check)

    # 2. 硬件连接
    hardware_check = _check_hardware_connections(app, config['channels'])
    checks.append(hardware_check)
```

```python
    # 3. 通信质量基线
    comm_check = _check_communication_baseline(app, config['test_id'])
    checks.append(comm_check)

    # 4. ECU 响应性
    ecu_check = _check_ecu_responsiveness(app, config['ecu_address'])
    checks.append(ecu_check)

    # 5. 系统资源
    resource_check = _check_system_resources()
    checks.append(resource_check)

    # 综合判定
    all_pass = all(c['passed'] for c in checks)
    critical_failures = [c for c in checks if not c['passed'] and c['critical']]

    return {
        'overall_status': 'PASS' if all_pass else 'FAIL',
        'can_proceed': len(critical_failures) == 0,
        'checks': checks,
        'recommendations': _generate_recommendations(checks)
    }
```

7. 开发步骤与实施建议

7.1 项目规划阶段

7.1.1 测试需求分析与范围定义 需求分析框架:

维度	分析内容	输出物
功能范围	需测试的 ECU 功能、诊断服务、网络接口	功能测试清单
协议覆盖	CAN/CAN FD/LIN/Ethernet/UDS/KWP2000	协议矩阵表
质量目标	代码覆盖率、需求覆盖率、缺陷密度目标	质量计划
环境约束	硬件资源、时间进度、人员技能	资源计划
合规要求	ISO 26262、ASPICE、网络安全等	合规检查清单

范围定义决策树:

```
是否需要 HIL 仿真?
├── 是 → 评估 Simulink/CarSim 集成需求
│        └── 是否需要实时性?
│              ├── 是 → 配置 dSPACE/NI PXI 实时平台
│              └── 否 → 基于 Windows 的软实时方案
└── 否 → 评估台架测试需求
         └── 是否需要真实负载?
               ├── 是 → 配置电子负载/电机模拟器
```

```
                    └── 否 → 纯通信测试方案
```

7.1.2 测试策略与工具选型 TSMaster 版本与授权选择:

版本	功能特性	适用场景	成本
TS Lite	基础分析，2 通道，无诊断/标定	学习评估、简单监测	免费/低 cost
TS Standard	完整通信，8 通道，UDS 诊断，C/Python 脚本	常规开发测试	中等
TS Professional	64 通道，XCP 标定，高级自动化，DoIP	完整开发验证	较高
TS Enterprise	浮动授权，多站点，定制开发支持	大型企业部署	协商

工具链集成决策:

集成需求	推荐方案	技术要点
测试管理	TSMaster + Polarion/Jama	ODX/ReqIF 导入，双向追溯
CI/CD	TSMaster + Jenkins/GitLab CI	COM API 命令行，JUnit 输出
需求追溯	TSMaster + DOORS/Integrity	自定义属性映射
缺陷跟踪	TSMaster + Jira/Azure DevOps	REST API 自动创建缺陷
数据管理	TSMaster + ASAM ODS/自定义	MDF4/MF4 标准格式

7.2 开发实施阶段

7.2.1 基础通信功能验证 分阶段验证路线图:

阶段	验证内容	通过标准	周期
W1	单通道收发、DBC 加载、信号解析	报文收发成功率 >99.9%	1 周
W2	多通道配置、周期报文、RBS 仿真	周期精度 <1%，抖动 <100 s	1 周
W3	LIN 主从、调度表、睡眠唤醒	调度时序误差 <5%	1 周
W4	Ethernet/DoIP 基础、大帧传输	TCP 连接稳定，传输完整	1-2 周

7.2.2 诊断服务逐步集成 诊断开发迭代计划:

迭代	服务内容	验证方法	交付物
Sprint 1	$10/$11 会话控制、$3E TesterPresent	状态机遍历测试	会话管理模块
Sprint 2	$27 安全访问（单级别）	Seed&Key 算法验证	安全访问模块

Table 35 – continued

迭代	服务内容	验证方法	交付物
Sprint 3	$22/$2E 数据读写（关键 DID）	边界值测试	数据服务模块
Sprint 4	$19/$14 DTC 管理	故障注入验证	DTC 模块
Sprint 5	$34-$37 刷写流程	完整刷写循环	Bootloader 模块
Sprint 6	多级别安全、完整 DID 覆盖、压力测试	自动化回归套件	完整诊断系统

7.2.3 自动化框架搭建 框架架构模式选择:

模式	特点	适用场景
数据驱动	测试数据与逻辑分离，Excel/JSON 驱动	大量相似用例，非技术人员维护
关键字驱动	封装业务关键字，组合成测试序列	跨项目复用，测试团队技能多样
行为驱动（BDD）	Given-When-Then 描述，利益相关者参与	需求澄清，验收标准明确
页面对象（PO）	封装 ECU 接口为对象，隐藏实现细节	多 ECU 变体，长期维护

Python pytest 集成示例:

```python
# conftest.py - pytest 配置和 fixture
import pytest
import win32com.client

@pytest.fixture(scope='session')
def tsmaster_app():
    """ 全局 TSMaster 应用实例"""
    app = win32com.client.Dispatch("TSMaster.Application")
    yield app
    app.Quit()

@pytest.fixture(scope='module')
def diagnostic_module(tsmaster_app, request):
    """ 模块级诊断接口"""
    ecu_config = request.module.ECU_CONFIG
    diag = tsmaster_app.GetDiagnostic()
    config = diag.GetConfig(ecu_config['name'])

    # 应用配置
    for key, value in ecu_config['parameters'].items():
        setattr(config, key, value)
    config.Apply()

    yield diag

    # 清理: 返回默认会话
    try:
```

```python
        diag.ExecuteService([0x10, 0x01])
    except:
        pass

# test_engine_ecu.py - 具体测试模块
ECU_CONFIG = {
    'name': 'EngineECU',
    'parameters': {
        'RequestID': 0x7E0,
        'ResponseID': 0x7E8,
        'P2Timeout': 50,
        'P2StarTimeout': 5000
    }
}

class TestSessionControl:
    """ 会话控制测试类"""

    def test_default_session_access(self, diagnostic_module):
        """ 验证默认会话下的服务可用性"""
        result = diagnostic_module.ExecuteService([0x10, 0x01])
        assert result['success'], f"进入默认会话失败: {result.get('nrc')}"

        # 验证默认会话专属服务可用
        vin_result = diagnostic_module.ExecuteService([0x22, 0xF1, 0x90])
        assert vin_result['success'], "默认会话应可读取 VIN"

    def test_programming_session_security(self, diagnostic_module):
        """ 验证编程会话的安全控制"""
        # 尝试直接进入编程会话（应失败或需要前置条件）
        result = diagnostic_module.ExecuteService([0x10, 0x02])

        # 预期：可能成功进入，但关键服务需要安全访问
        if result['success']:
            # 验证未解锁时受保护服务被拒绝
            write_result = diagnostic_module.ExecuteService(
                [0x2E, 0xF1, 0x90] + [0x00]*17
            )
            assert not write_result['success'], "未解锁应拒绝写操作"
            assert write_result.get('nrc') == 0x33, "应返回 SecurityAccessDenied"

# 执行: pytest test_engine_ecu.py -v --html=report.html
```

7.3 验证部署阶段

7.3.1 测试用例评审与优化 评审检查清单：

检查项	检查内容	优化方向
需求覆盖	每个需求至少有一个正向测试	补充缺失覆盖
边界覆盖	等价类边界、特殊值、非法值	增加边界用例

Table 37 – continued

检查项	检查内容	优化方向
错误处理	否定响应、超时、异常流程	补充负面测试
可重复性	无外部依赖，结果确定	消除随机因素
执行效率	用例执行时间，资源占用	并行化，减少等待
可维护性	代码结构清晰，注释完整	重构，提取公共函数
可追溯性	需求 用例 代码 结果关联	完善元数据标注

7.3.2 持续集成与回归测试 CI 流水线配置（GitLab CI 示例）：

```yaml
# .gitlab-ci.yml
stages:
  - build
  - unit-test
  - integration-test
  - system-test
  - report

variables:
  TSM_LICENSE_SERVER: "license.tosun.internal:27000"

before_script:
  - pip install -r requirements.txt
  - python -c "import win32com.client; print('COM ready')"

unit-test:
  stage: unit-test
  script:
    - pytest tests/unit/ -v --cov=src --cov-report=xml
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml

can-communication-test:
  stage: integration-test
  tags: [tsmaster-hardware]
  script:
    - python -m pytest tests/integration/test_can_communication.py -v
  artifacts:
    reports:
      junit: junit-can.xml

uds-diagnostic-test:
  stage: integration-test
  tags: [tsmaster-hardware, ecu-bench]
  script:
    - python -m pytest tests/integration/test_uds_services.py -v --ecu-config=configs/engine_ecu.json
  parallel:
```

```
    matrix:
      - ECU_VARIANT: [BASE, PREMIUM, SPORT]
  artifacts:
    reports:
      junit: junit-uds-${ECU_VARIANT}.xml

nightly-regression:
  stage: system-test
  tags: [tsmaster-hardware, full-rig]
  only:
    - schedules
  script:
    - python run_regression_suite.py --full --report=html
  artifacts:
    name: "nightly-report-$CI_COMMIT_SHORT_SHA"
    paths:
      - reports/
    expire_in: 30 days

merge-report:
  stage: report
  script:
    - python merge_reports.py --input reports/ --output final-report/
  artifacts:
    paths:
      - final-report/
```

7.3.3 维护更新与版本管理 版本管理策略：

资产类型	管理工具	版本策略	分支策略
TSMaster 工程文件	Git LFS	语义化版本（主. 次. 修）	功能分支，主干发布
Python 测试代码	Git	与 ECU 软件版本对齐	Git Flow
DBC/ODX 数据库	专用工具（CANdb++）	与网络设计版本关联	变更审批流程
测试数据（BLF）	对象存储	日期 + 测试运行 ID	生命周期管理
测试报告	文档管理系统	与测试执行绑定	不可变存储

TSMaster 版本兼容性管理：

- **工程文件版本**：TSMaster 自动升级，向后兼容警告
- **API 接口版本**：COM 接口稳定，重大变更文档化
- **脚本代码版本**：Python 2→3 迁移经验，类型注解
- **硬件驱动版本**：与 TSMaster 版本绑定，同步更新